

SERVER_IMPLEMENTATION

Helix OTA Server Implementation Design

Version: 1.0.0-MVP **Status:** Design Specification **Last Updated:** 2025-03-04

Table of Contents

- 1. [Project Structure](#)
 - 2. [Go Module Definition](#)
 - 3. [Core Interfaces](#)
 - 4. [Rollout Engine](#)
 - 5. [Artifact Validation Pipeline](#)
 - 6. [Device Registration & Authentication](#)
 - 7. [Telemetry Collection & Analysis](#)
 - 8. [Background Jobs](#)
 - 9. [Middleware Chain](#)
 - 10. [Server Main Implementation](#)
 - 11. [Integration with vasic-digital Submodules](#)
-

1. Project Structure

The Helix OTA server follows the standard Go project layout convention, enforcing strict boundary separation between public and internal packages. Every import path crosses a clearly defined interface, making the codebase testable, modular, and resistant to circular dependencies.

```
helix-ota-server/
├── cmd/
│   └── server/
│       └── main.go                                # Application entry point
├── internal/
│   ├── config/
│   │   └── config.go                            # Configuration loading &
validation
│   └── config_test.go
└── handler/
```

			update_handler.go	# HTTP handlers for update
endpoints				
			device_handler.go	# HTTP handlers for device
endpoints				
			rollout_handler.go	# HTTP handlers for rollout
endpoints				
			artifact_handler.go	# HTTP handlers for
artifact endpoints				
			telemetry_handler.go	# HTTP handlers for
telemetry endpoints				
			auth_handler.go	# HTTP handlers for auth
endpoints				
			handler_test.go	
			service/	
			update_service.go	# Update business logic
			device_service.go	# Device management
business logic				
			rollout_service.go	# Rollout orchestration
business logic				
			rollout_engine.go	# Rollout decision engine
(CRITICAL)				
			artifact_service.go	# Artifact management
business logic				
			telemetry_service.go	# Telemetry processing
business logic				
			auth_service.go	# Authentication business
logic				
			service_test.go	
			repository/	
			postgres/	
			update_repo.go	# PostgreSQL update
repository				
			device_repo.go	# PostgreSQL device
repository				
			rollout_repo.go	# PostgreSQL rollout
repository				
			artifact_repo.go	# PostgreSQL artifact
repository				
			telemetry_repo.go	# PostgreSQL telemetry
repository				
			redis/	
			device_cache.go	# Redis device cache
			rollout_cache.go	# Redis rollout state cache
			rate_limiter_store.go	# Redis-backed rate limit
store				
			repo_test.go	
			model/	
			update.go	# Update domain model
			device.go	# Device domain model
			rollout.go	# Rollout domain model
			artifact.go	# Artifact domain model
			telemetry.go	# Telemetry event domain
model				
			user.go	# User domain model

		└ errors.go	# Domain-specific error
types			
		└ middleware/	
		└ chain.go	# Middleware chain builder
		└ auth.go	# JWT + mTLS auth
middleware			
		└ rbac.go	# Role-based access control
		└ logging.go	# Request/response logging
		└ request_id.go	# Request ID tracing
		└ cors.go	# CORS configuration
		└ middleware_test.go	
		└ validation/	
		└ pipeline.go	# Validation pipeline
orchestrator			
		└ structure.go	# Artifact structure
validation			
		└ hash.go	# Hash verification
(streaming)			
		└ signature.go	# RSA signature
verification			
		└ compatibility.go	# Device compatibility
check			
		└ worker_pool.go	# Concurrent validation
workers			
		└ validation_test.go	
└ pkg/			
		└ api/	
		└ requests.go	# API request types
		└ responses.go	# API response types
		└ errors.go	# API error types
		└ auth/	
		└ jwt.go	# JWT token management
		└ mtls.go	# mTLS certificate handling
		└ device_identity.go	# Device ID generation
		└ telemetry/	
		└ metrics.go	# Metrics type definitions
		└ events.go	# Event type definitions
		└ anomaly.go	# Anomaly detection types
└ migrations/			
		└ 001_create_devices.up.sql	
		└ 001_create_devices.down.sql	
		└ 002_create_artifacts.up.sql	
		└ 002_create_artifacts.down.sql	
		└ 003_create_updates.up.sql	
		└ 003_create_updates.down.sql	
		└ 004_create_rollouts.up.sql	
		└ 004_create_rollouts.down.sql	
		└ 005_create_telemetry.up.sql	
		└ 005_create_telemetry.down.sql	
		└ 006_create_users.up.sql	
		└ 006_create_users.down.sql	
└ go.mod			
└ go.sum			
└ Makefile			

Key Architectural Decisions

Internal vs. Pkg Boundary: The `internal/` directory prevents external Go projects from importing implementation details. Only `pkg/` types are importable by other modules. Handlers never import repositories directly — they call services, which call repositories. This three-layer architecture (handler → service → repository) ensures each layer can be mocked independently for testing.

No Global State: All server state flows through explicit dependency injection. The main function wires dependencies together and passes them down. There are no package-level `var db *sql.DB` patterns.

Error Propagation: Domain errors defined in `internal/model/errors.go` bubble up through service and handler layers. Handlers map domain errors to HTTP status codes using a centralized mapper, ensuring consistent API error responses.

2. Go Module Definition

```
// go.mod
module github.com/vasic-digital/helix-ota-server

go 1.22

require (
    // — vasic-digital internal submodules —
    digital.vasic/auth          v1.2.0
    digital.vasic/database      v1.3.1
    digital.vasic/cache         v1.1.0
    digital.vasic/observability v1.0.4
    digital.vasic/security      v1.1.2
    digital.vasic/middleware    v1.2.1
    digital.vasic/config        v1.0.3
    digital.vasic/eventbus      v1.1.0
    digital.vasic/storage       v1.2.0
    digital.vasic/ratelimiter   v1.0.2
    digital.vasic/concurrency   v1.0.1
    digital.vasic/recovery      v1.0.0
    digital.vasic/backgroundtasks v1.1.0

    // — HTTP framework —
    github.com/go-chi/chi/v5    v5.0.12

    // — Database —
    github.com/jackc/pgx/v5      v5.5.3
    github.com/golang-migrate/migrate/v4 v4.17.0

    // — Validation —
```

```

github.com/go-playground/validator/v10 v10.17.0

// — Cryptography —
github.com/lestrrat-go/jwx/v2 v2.0.19

// — Observability —
go.opentelemetry.io/otel v1.22.0
go.opentelemetry.io/otel/trace v1.22.0

// — Utilities —
github.com/google/uuid v1.6.0
github.com/robfig/cron/v3 v3.0.1
go.uber.org/zap v1.26.0
golang.org/x/crypto v0.18.0
golang.org/x/sync v0.6.0
)

require (
    // Indirect dependencies omitted for brevity – generated by
    // `go mod tidy`
    github.com/jackc/pgx/v5/stdlib v5.5.3 // indirect
    github.com/jackc/pgx/v5/pgxpool v5.5.3 // indirect
    github.com/redis/go-redis/v9 v9.4.0 // indirect
    github.com/prometheus/client_golang v1.18.0 // indirect
)

```

Module Versioning Strategy

All vasic-digital submodules follow semantic versioning. The `go.mod` pins to minor versions (e.g., `v1.2.0`) allowing patch updates via `go get -u=patch` but preventing breaking changes. Release CI enforces that every submodule bump passes the full integration test suite before merging.

3. Core Interfaces

All service interfaces are defined in their respective service files. Interfaces are consumed by handlers (above) and implemented by concrete service structs (below). Repository interfaces are defined within the service package to follow the Dependency Inversion Principle — the service defines what it needs, not what the repository provides.

3.1 UpdateService

```
package service
```

```

import (
    "context"
    "time"

    "github.com/vasic-digital/helix-ota-server/internal/model"
)

```

```

    "github.com/vasic-digital/helix-ota-server/pkg/api"
)

// UpdateService defines the contract for update operations.
// Implementations must be safe for concurrent use.
type UpdateService interface {
    // CheckForUpdate determines whether an update is available
    // for the given
    // device based on its current version, hardware revision, and
    // rollout
    // eligibility. Returns nil if no update is available.
    CheckForUpdate(ctx context.Context, req
        *api.CheckUpdateRequest) (*api.UpdateAvailableResponse,
            error)

    // GetUpdate retrieves full details of a specific update
    // including
    // associated artifact metadata and rollout status.
    GetUpdate(ctx context.Context, updateID string)
        (*model.Update, error)

    // ListUpdates returns a paginated list of updates, optionally
    // filtered
    // by target version, hardware revision, or status.
    ListUpdates(ctx context.Context, filter *api.UpdateFilter,
        page *api.Pagination)
        (*api.PaginatedResult[model.Update], error)
}

// CheckUpdateRequest contains the device context needed to
// determine
// update eligibility.
type CheckUpdateRequest struct {
    DeviceID      string
    CurrentVersion string
    HardwareRevision string
    DeviceGroup   string
    TargetVersion  string // Optional: device requests specific
        version
}

```

3.2 DeviceService

```

// DeviceService defines the contract for device lifecycle
// management.
// Device operations are idempotent where possible to support
// retry semantics.
type DeviceService interface {
    // Register enrolls a new device in the system. Generates a
    // unique device
    // ID from hardware properties and initiates mTLS certificate
    // enrollment.
    // Returns ErrDeviceAlreadyExists if the device fingerprint
    // matches an

```

```

// existing record.
Register(ctx context.Context, req *api.RegisterDeviceRequest)
    (*model.Device, error)

// Get retrieves a device by its ID. Returns ErrDeviceNotFound
// if the
// device does not exist.
Get(ctx context.Context, deviceID string) (*model.Device,
    error)

// List returns a paginated list of devices with optional
// filtering by
// status, group, hardware revision, or last-seen threshold.
List(ctx context.Context, filter *api.DeviceFilter, page
    *api.Pagination) (*api.PaginatedResult[model.Device],
    error)

// Update modifies device metadata (tags, group, display
// name).
// Does NOT update hardware properties – those are immutable
// after registration.
Update(ctx context.Context, deviceID string, req
    *api.UpdateDeviceRequest) (*model.Device, error)

// Decommission marks a device as decommissioned. The device
// will no longer
// receive updates. Its mTLS certificate is revoked. Telemetry
// data is
// retained for audit purposes.
Decommission(ctx context.Context, deviceID string) error
}

```

3.3 RolloutService

```

// RolloutService defines the contract for managing update
// rollouts.
// Rollouts are state machines with strict transition rules.
type RolloutService interface {
    // Create initializes a new rollout. Validates that the
    // referenced update
    // and artifact exist, and that no overlapping rollout targets
    // the same
    // device group for the same hardware revision.
    Create(ctx context.Context, req *api.CreateRolloutRequest)
        (*model.Rollout, error)

    // Get retrieves a rollout by ID including current progress
    // metrics.
    Get(ctx context.Context, rolloutID string) (*model.Rollout,
        error)

    // List returns paginated rollouts with optional status
    // filtering.

```

```

List(ctx context.Context, filter *api.RolloutFilter, page
    *api.Pagination) (*api.PaginatedResult[model.Rollout],
    error)

// Update modifies rollout parameters (stages, failure
// threshold, description).
// Only valid when rollout is in PAUSED or DRAFT state.
Update(ctx context.Context, rolloutID string, req
    *api.UpdateRolloutRequest) (*model.Rollout, error)

// Pause halts a rollout. No new devices will be assigned the
// update.
// Devices already downloading or installing continue
// uninterrupted.
Pause(ctx context.Context, rolloutID string) error

// Resume continues a paused rollout from its current stage
// and percentage.
Resume(ctx context.Context, rolloutID string) error

// Halt permanently stops a rollout. Unlike pause, halt cannot
// be resumed.
// Optionally triggers auto-rollback if configured.
Halt(ctx context.Context, rolloutID string, reason string)
    error

// GetProgress returns real-time rollout progress metrics
// including
// devices completed, failed, in-progress, and pending.
GetProgress(ctx context.Context, rolloutID string)
    (*api.RolloutProgress, error)
}

```

3.4 ArtifactService

```

// ArtifactService defines the contract for OTA artifact lifecycle
// management.
// Artifacts are immutable once validated – they can only be
// deleted, never modified.
type ArtifactService interface {
    // Upload receives an artifact stream, persists it to object
    // storage, and
    // enqueues it for validation. Returns immediately with a
    // pending artifact
    // record; validation proceeds asynchronously.
    Upload(ctx context.Context, req *api.UploadArtifactRequest)
        (*model.Artifact, error)

    // Validate triggers the full validation pipeline on an
    // artifact.
    // Normally called automatically after upload, but can be
    // invoked manually
    // for re-validation after a validation rule change.
    Validate(ctx context.Context, artifactID string)
        (*api.ValidationResult, error)
}

```



```

// Get retrieves artifact metadata by ID.
Get(ctx context.Context, artifactID string) (*model.Artifact,
    error)

// List returns paginated artifacts with optional filtering by
    type,
// target version, hardware revision, or validation status.
List(ctx context.Context, filter *api.ArtifactFilter, page
    *api.Pagination) (*api.PaginatedResult[model.Artifact],
    error)

// Delete removes an artifact from storage and metadata. Fails
    if the
// artifact is referenced by any active rollout.
Delete(ctx context.Context, artifactID string) error

// Download returns a signed, time-limited URL for artifact
    download.
// The URL expires after the configured TTL (default: 1 hour).
Download(ctx context.Context, artifactID string)
    (*api.DownloadURL, error)
}

```

3.5 TelemetryService

```

// TelemetryService defines the contract for device telemetry
    ingestion and analysis.
// Telemetry is write-heavy – millions of events per hour – and
    read patterns
// favor aggregated metrics over individual event retrieval.
type TelemetryService interface {
    // ReportEvent ingests a single telemetry event from a device.
    // Events are validated, enriched with server-side metadata
        (timestamp,
    // region), and published to the event bus for async
        processing.
    ReportEvent(ctx context.Context, event
        *api.TelemetryEventRequest) error

    // GetOverview returns aggregated telemetry metrics across all
        devices
    // or filtered by rollout, update, or device group.
    GetOverview(ctx context.Context, filter *api.TelemetryFilter)
        (*api.TelemetryOverview, error)

    // GetDeviceTelemetry returns telemetry events for a specific
        device,
    // paginated and ordered by timestamp descending.
    GetDeviceTelemetry(ctx context.Context, deviceID string,
        filter *api.TelemetryFilter, page *api.Pagination)
        (*api.PaginatedResult[model.TelemetryEvent], error)
}

```

3.6 AuthService

```
// AuthService defines the contract for authentication and token
// management.
// Supports two authentication modes: user credentials (JWT) and
// device
// certificates (mTLS).
type AuthService interface {
    // Login authenticates a user with email/password credentials.
    // Returns access and refresh JWT tokens on success.
    Login(ctx context.Context, req *api.LoginRequest)
        (*api.TokenPair, error)

    // RefreshToken exchanges a valid refresh token for a new
    // token pair.
    // The old refresh token is invalidated (rotation).
    RefreshToken(ctx context.Context, refreshToken string)
        (*api.TokenPair, error)

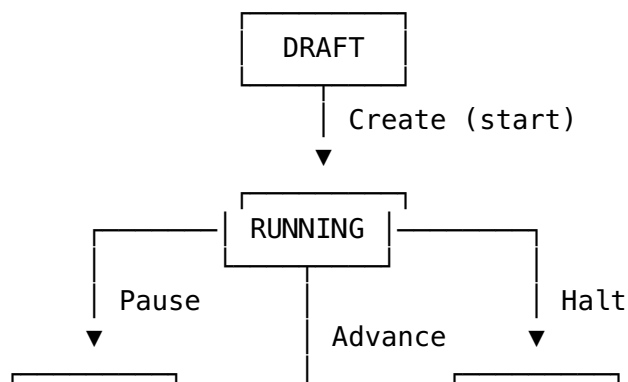
    // ValidateToken verifies a JWT access token and returns the
    // claims.
    // Checks signature, expiration, and token revocation status.
    ValidateToken(ctx context.Context, accessToken string)
        (*api.TokenClaims, error)

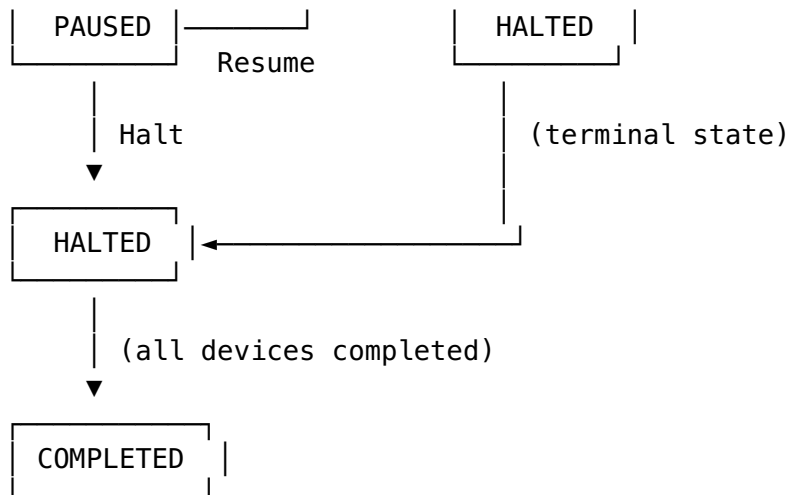
    // RegisterDevice creates device credentials after mTLS
    // certificate
    // verification. Generates a device-scoped JWT for API access.
    RegisterDevice(ctx context.Context, certPEM []byte)
        (*api.DeviceCredentials, error)
}
```

4. Rollout Engine (CRITICAL)

The rollout engine is the most complex component of the Helix OTA server. It controls how updates are progressively delivered to devices, monitors health signals, and makes autonomous decisions about advancing, pausing, or halting rollouts.

4.1 Rollout State Machine





4.2 Rollout Domain Model

```
package model
```

```
import (
    "time"
)
```

```
type RolloutStatus string
```

```
const (
    RolloutStatusDraft      RolloutStatus = "DRAFT"
    RolloutStatusRunning    RolloutStatus = "RUNNING"
    RolloutStatusPaused     RolloutStatus = "PAUSED"
    RolloutStatusHalted     RolloutStatus = "HALTED"
    RolloutStatusCompleted RolloutStatus = "COMPLETED"
)
```

```
type Rollout struct {
    ID            string    `json:"id" db:"id"`
    UpdateID      string    `json:"update_id" db:"update_id"`
    ArtifactID    string    `json:"artifact_id" db:"artifact_id"`
    Name          string    `json:"name" db:"name"`
    Description   string    `json:"description" db:"description"`
    Status        RolloutStatus `json:"status" db:"status"`
    TargetGroup   string    `json:"target_group" db:"target_group"`
    HardwareRev   string    `json:"hardware_rev" db:"hardware_rev"`
}
```

```
// Stages defines the progressive rollout schedule.
// Each stage has a target percentage and a minimum dwell time
// before advancing to the next stage.
Stages []RolloutStage `json:"stages" db:"stages"`
```

```

    // CurrentStageIndex tracks which stage the rollout is
    // currently in.
    CurrentStageIndex int          `json:"current_stage_index"
    db:"current_stage_index"`

    // FailureThreshold is the maximum allowable failure rate (0.0
    // - 1.0)
    // before auto-rollback is triggered.
    FailureThreshold float64       `json:"failure_threshold"
    db:"failure_threshold"`

    // MinDwellDuration is the minimum time a stage must run
    // before
    // the engine considers advancing to the next stage.
    MinDwellDuration time.Duration `json:"min_dwell_duration"
    db:"min_dwell_duration"`

    // StageEnteredAt records when the rollout entered the current
    // stage.
    StageEnteredAt *time.Time      `json:"stage_entered_at"
    db:"stage_entered_at"`

    CreatedBy      string          `json:"created_by"
    db:"created_by"`
    CreatedAt      time.Time       `json:"created_at"
    db:"created_at"`
    UpdatedAt      time.Time       `json:"updated_at"
    db:"updated_at"`
    HaltedReason   string          `json:"halted_reason,omitempty"
    db:"halted_reason"`
}

type RolloutStage struct {
    Percentage      float64          `json:"percentage"`
    MinDwellDuration time.Duration `json:"min_dwell_duration"`
}

// DefaultStages returns the standard progressive rollout
// schedule:
// 5% → 10% → 30% → 50% → 100%
func DefaultStages() []RolloutStage {
    return []RolloutStage{
        {Percentage: 5.0, MinDwellDuration: 30 * time.Minute},
        {Percentage: 10.0, MinDwellDuration: 1 * time.Hour},
        {Percentage: 30.0, MinDwellDuration: 4 * time.Hour},
        {Percentage: 50.0, MinDwellDuration: 12 * time.Hour},
        {Percentage: 100.0, MinDwellDuration: 0}, // terminal
        stage
    }
}

```

4.3 Device Selection Algorithm

The device selection algorithm determines which devices receive the update at each rollout stage. It uses deterministic hashing to ensure consistent device selection across restarts while supporting percentage-based targeting.

```
package service

import (
    "crypto/sha256"
    "encoding/binary"
    "fmt"
    "sort"
)

// DeviceSelector determines which devices are eligible for a
// rollout
// at a given percentage. Uses deterministic hashing so that the
// same
// device always maps to the same bucket regardless of server
// restarts.
type DeviceSelector struct{}

// NewDeviceSelector creates a new DeviceSelector instance.
func NewDeviceSelector() *DeviceSelector {
    return &DeviceSelector{}
}

// SelectForPercentage returns the subset of devices that fall
// within
// the given rollout percentage. The algorithm:
// 1. Sort devices by a stable key (device ID) for determinism.
// 2. Hash each device ID combined with the rollout ID to
//    produce
//    a uniformly distributed value in [0, 100).
// 3. Include devices whose hash value is less than the target
//    percentage.
//
// This ensures:
// - The same device always gets the same hash for a given
//   rollout.
// - Increasing the percentage always includes all previously
//   selected
//   devices plus new ones (monotonic inclusion).
// - The selection is uniformly distributed across the device
//   fleet.
func (s *DeviceSelector) SelectForPercentage(
    devices []DeviceCandidate,
    rolloutID string,
    percentage float64,
) []DeviceCandidate {
    // Sort by device ID for deterministic ordering
    sort.Slice(devices, func(i, j int) bool {
        return devices[i].ID < devices[j].ID
    })
}
```

```

    })

    var selected []DeviceCandidate
    for _, device := range devices {
        bucket := s.hashToBucket(device.ID, rolloutID)
        if bucket < percentage {
            selected = append(selected, device)
        }
    }
    return selected
}

// hashToBucket produces a value in [0, 100) by hashing the device
// ID
// and rollout ID together. Uses SHA-256 for uniform distribution.
func (s *DeviceSelector) hashToBucket(deviceID, rolloutID string)
    float64 {
    h := sha256.New()
    h.Write([]byte(fmt.Sprintf("%s:%s", rolloutID, deviceID)))
    digest := h.Sum(nil)

    // Take the first 8 bytes and convert to a uint64
    val := binary.BigEndian.Uint64(digest[:8])

    // Map to [0, 100) using modular arithmetic on the float64
    // range
    // This gives us a nearly uniform distribution.
    return float64(val%10000) / 100.0
}

// DeviceCandidate represents a device eligible for a rollout.
type DeviceCandidate struct {
    ID                string
    CurrentVersion    string
    HardwareRevision  string
    Group             string
    LastSeenAt        int64 // Unix timestamp
}

```

4.4 Rollout Decision Engine

The rollout decision engine runs as a background scheduler. On each tick, it evaluates all active rollouts and decides whether to advance to the next stage, pause for health monitoring, or halt due to excessive failures.

```
package service
```

```
import (
    "context"
    "fmt"
    "math"
    "sync"
    "time"

```

```

    "digital.vasic/backgroundtasks"
    "digital.vasic/observability"

    "github.com/vasic-digital/helix-ota-server/internal/model"
    "github.com/vasic-digital/helix-ota-server/pkg/telemetry"
)

// RolloutDecisionEngine evaluates active rollouts and makes
//    progression
// decisions based on health signals and time-based criteria.
// It is the core autonomous controller of the rollout system.
type RolloutDecisionEngine struct {
    rolloutRepo    RolloutRepository
    deviceRepo     DeviceRepository
    telemetryRepo  TelemetryRepository
    eventBus       EventPublisher
    selector       *DeviceSelector
    mu             sync.Mutex
    logger         *observability.Logger
    metrics        *rolloutMetrics
}

// NewRolloutDecisionEngine creates a new decision engine with all
//    dependencies.
func NewRolloutDecisionEngine(
    rolloutRepo RolloutRepository,
    deviceRepo DeviceRepository,
    telemetryRepo TelemetryRepository,
    eventBus EventPublisher,
    logger *observability.Logger,
) *RolloutDecisionEngine {
    return &RolloutDecisionEngine{
        rolloutRepo:    rolloutRepo,
        deviceRepo:     deviceRepo,
        telemetryRepo:  telemetryRepo,
        eventBus:       eventBus,
        selector:       NewDeviceSelector(),
        logger:         logger,
        metrics:        newRolloutMetrics(),
    }
}

// EvaluateAll is the main evaluation loop. Called periodically by
//    the
// background task scheduler. It iterates all RUNNING rollouts and
//    applies the decision logic to each one.
func (e *RolloutDecisionEngine) EvaluateAll(ctx context.Context)
    error {
    rollouts, err := e.rolloutRepo.ListByStatus(ctx,
        model.RolloutStatusRunning)
    if err != nil {
        return fmt.Errorf("list running rollouts: %w", err)
    }
}

```

```

    }

    var evalErrors []error
    for _, rollout := range rollouts {
        if err := e.evaluateOne(ctx, rollout); err != nil {
            e.logger.Error("rollout evaluation failed",
                "rollout_id", rollout.ID,
                "error", err,
            )
            evalErrors = append(evalErrors, err)
        }
    }

    if len(evalErrors) > 0 {
        return fmt.Errorf("%d rollout evaluations failed: %v",
            len(evalErrors), evalErrors[0])
    }
    return nil
}

// evaluateOne applies the full decision logic to a single
// rollout.
// The decision tree:
//
// 1. Is the rollout in a terminal stage (100%) and all devices
//    completed?
//    → Mark as COMPLETED.
//
// 2. Has the minimum dwell time for the current stage elapsed?
//    → If not, skip (remain in current stage).
//
// 3. Is the failure rate above the threshold?
//    → HALT the rollout and publish a RolloutHalted event.
//
// 4. Is the success rate healthy and dwell time met?
//    → Advance to the next stage.
//
func (e *RolloutDecisionEngine) evaluateOne(ctx context.Context,
    rollout *model.Rollout) error {
    // Acquire per-rollout lock to prevent concurrent evaluation
    e.mu.Lock()
    defer e.mu.Unlock()

    // Step 1: Check if rollout is complete
    progress, err := e.computeProgress(ctx, rollout)
    if err != nil {
        return fmt.Errorf("compute progress for rollout %s: %w",
            rollout.ID, err)
    }

    if e.isComplete(progress, rollout) {
        return e.completeRollout(ctx, rollout)
    }
}

```



```

// Step 2: Check minimum dwell time
if !e.dwellTimeElapsed(rollout) {
    e.logger.Debug("dwell time not elapsed",
        "rollout_id", rollout.ID,
        "stage", rollout.CurrentStageIndex,
        "stage_entered_at", rollout.StageEnteredAt,
    )
    return nil
}

// Step 3: Check failure rate for auto-rollback
failureRate := e.computeFailureRate(progress)
if failureRate > rollout.FailureThreshold {
    e.logger.Warn("failure rate exceeded threshold, halting
        rollout",
        "rollout_id", rollout.ID,
        "failure_rate", fmt.Sprintf("%.2f%%",
            failureRate*100),
        "threshold", fmt.Sprintf("%.2f%%",
            rollout.FailureThreshold*100),
    )
    return e.haltRollout(ctx, rollout, fmt.Sprintf(
        "auto-rollback: failure rate %.1f%% exceeded threshold
        %.1f%%",
        failureRate*100, rollout.FailureThreshold*100,
    ))
}

// Step 4: Advance to next stage if healthy
if e.canAdvance(rollout) {
    return e.advanceStage(ctx, rollout, progress)
}

return nil
}

// computeProgress queries telemetry data to compute current
// rollout metrics.
func (e *RolloutDecisionEngine) computeProgress(
    ctx context.Context,
    rollout *model.Rollout,
) (*RolloutProgressMetrics, error) {
    // Get all devices eligible for this rollout
    devices, err := e.deviceRepo.ListByGroupAndHardware(
        ctx, rollout.TargetGroup, rollout.HardwareRev,
    )
    if err != nil {
        return nil, fmt.Errorf("list eligible devices: %w", err)
    }

    // Get telemetry for this rollout in the current stage window

```

```

stats, err := e.telemetryRepo.GetRolloutStats(ctx,
    rollout.ID, rollout.StageEnteredAt)
if err != nil {
    return nil, fmt.Errorf("get rollout stats: %w", err)
}

currentStage := rollout.Stages[rollout.CurrentStageIndex]
targetDevices := e.selector.SelectForPercentage(
    devices, rollout.ID, currentStage.Percentage,
)

return &RolloutProgressMetrics{
    TotalEligibleDevices: len(devices),
    TargetDeviceCount:    len(targetDevices),
    DevicesCompleted:     stats.CompletedCount,
    DevicesFailed:        stats.FailedCount,
    DevicesInProgress:    stats.InProgressCount,
    DevicesPending:       len(targetDevices) -
        stats.CompletedCount - stats.FailedCount -
        stats.InProgressCount,
    FailureRate:          stats.FailureRate(),
    SuccessRate:          stats.SuccessRate(),
    AverageDownloadTime:  stats.AvgDownloadDuration,
    AverageInstallTime:   stats.AvgInstallDuration,
}, nil
}

// computeFailureRate returns the failure rate as a fraction (0.0
// - 1.0).
// Only considers devices that have reached a terminal state
// (completed or failed).
func (e *RolloutDecisionEngine) computeFailureRate(progress
    *RolloutProgressMetrics) float64 {
    total := progress.DevicesCompleted + progress.DevicesFailed
    if total == 0 {
        return 0.0
    }
    return float64(progress.DevicesFailed) / float64(total)
}

// dwellTimeElapsed checks if the minimum dwell time for the
// current stage
// has passed since the rollout entered this stage.
func (e *RolloutDecisionEngine) dwellTimeElapsed(rollout
    *model.Rollout) bool {
    if rollout.StageEnteredAt == nil {
        return false
    }
    currentStage := rollout.Stages[rollout.CurrentStageIndex]
    elapsed := time.Since(*rollout.StageEnteredAt)
    return elapsed >= currentStage.MinDwellDuration
}

// canAdvance checks whether there is a next stage to advance to.

```

```

func (e *RolloutDecisionEngine) canAdvance(rollout
    *model.Rollout) bool {
    return rollout.CurrentStageIndex < len(rollout.Stages)-1
}

// advanceStage moves the rollout to the next stage and updates
device assignments.
func (e *RolloutDecisionEngine) advanceStage(
    ctx context.Context,
    rollout *model.Rollout,
    progress *RolloutProgressMetrics,
) error {
    nextIndex := rollout.CurrentStageIndex + 1
    nextStage := rollout.Stages[nextIndex]
    now := time.Now().UTC()

    rollout.CurrentStageIndex = nextIndex
    rollout.StageEnteredAt = &now
    rollout.UpdatedAt = now

    if err := e.rolloutRepo.Update(ctx, rollout); err != nil {
        return fmt.Errorf("update rollout stage: %w", err)
    }

    // Publish event for other services (telemetry, notifications)
    e.eventBus.Publish(ctx, Event{
        Type:    "rollout.stage_advanced",
        Payload: map[string]interface{}{
            "rollout_id":    rollout.ID,
            "new_stage":      nextIndex,
            "new_percentage": nextStage.Percentage,
        },
    })

    e.logger.Info("rollout advanced to next stage",
        "rollout_id", rollout.ID,
        "new_stage", nextIndex,
        "new_percentage", fmt.Sprintf("%.1f%%",
            nextStage.Percentage),
        "completed_devices", progress.DevicesCompleted,
        "failed_devices", progress.DevicesFailed,
    )

    e.metrics.stageAdvances.Inc()
    return nil
}

// haltRollout stops a rollout and publishes a halt event.
func (e *RolloutDecisionEngine) haltRollout(
    ctx context.Context,
    rollout *model.Rollout,
    reason string,
) error {

```

```

now := time.Now().UTC()
rollout.Status = model.RolloutStatusHalted
rollout.HaltedReason = reason
rollout.UpdatedAt = now

if err := e.rolloutRepo.Update(ctx, rollout); err != nil {
    return fmt.Errorf("halt rollout: %w", err)
}

e.eventBus.Publish(ctx, Event{
    Type:    "rollout.halted",
    Payload: map[string]interface{}{
        "rollout_id": rollout.ID,
        "reason":     reason,
    },
})

e.metrics.autoHalts.Inc()
return nil
}

// completeRollout marks a rollout as fully completed.
func (e *RolloutDecisionEngine) completeRollout(
    ctx context.Context,
    rollout *model.Rollout,
) error {
    now := time.Now().UTC()
    rollout.Status = model.RolloutStatusCompleted
    rollout.UpdatedAt = now

    if err := e.rolloutRepo.Update(ctx, rollout); err != nil {
        return fmt.Errorf("complete rollout: %w", err)
    }

    e.eventBus.Publish(ctx, Event{
        Type:    "rollout.completed",
        Payload: map[string]interface{}{"rollout_id": rollout.ID},
    })

    e.logger.Info("rollout completed", "rollout_id", rollout.ID)
    return nil
}

// isComplete determines if a rollout has reached completion:
// the current stage is 100% and all targeted devices have
// finished.
func (e *RolloutDecisionEngine) isComplete(
    progress *RolloutProgressMetrics,
    rollout *model.Rollout,
) bool {
    currentStage := rollout.Stages[rollout.CurrentStageIndex]
    if currentStage.Percentage < 100.0 {
        return false
    }

```

```

    }
    return progress.DevicesPending == 0 &&
        progress.DevicesInProgress == 0
}

// RolloutProgressMetrics holds computed metrics for a rollout at
// a point in time.
type RolloutProgressMetrics struct {
    TotalEligibleDevices int
        `json:"total_eligible_devices"`
    TargetDeviceCount    int
        `json:"target_device_count"`
    DevicesCompleted     int
        `json:"devices_completed"`
    DevicesFailed        int
        `json:"devices_failed"`
    DevicesInProgress    int
        `json:"devices_in_progress"`
    DevicesPending       int
        `json:"devices_pending"`
    FailureRate          float64
        `json:"failure_rate"`
    SuccessRate          float64
        `json:"success_rate"`
    AverageDownloadTime  time.Duration
        `json:"average_download_time"`
    AverageInstallTime   time.Duration
        `json:"average_install_time"`
}

// rolloutMetrics tracks Prometheus-style counters for the
// decision engine.
type rolloutMetrics struct {
    stageAdvances *observability.Counter
    autoHalts     *observability.Counter
    evaluations   *observability.Counter
}

func newRolloutMetrics() *rolloutMetrics {
    return &rolloutMetrics{
        stageAdvances:
            observability.NewCounter("rollout_stage_advances_total"),
        autoHalts:
            observability.NewCounter("rollout_auto_halts_total"),
        evaluations:
            observability.NewCounter("rollout_evaluations_total"),
    }
}

```

4.5 Concurrency and Race Conditions

Multiple rollouts can target overlapping device sets. The decision engine must handle these scenarios:

```

// RolloutConcurrencyGuard prevents conflicting device assignments
// when multiple rollouts target the same device group.
type RolloutConcurrencyGuard struct {
    cache DistributedCache
}

```

```

// AcquireDeviceAssignment attempts to assign a device to a
// rollout.
// Uses an atomic Redis SETNX operation to ensure only one rollout
// can claim a device at a time.
//
// The key pattern is: device_assignment:{device_id}
// The value is the rollout ID that claimed the device.
// TTL is set to 24 hours to prevent permanent locks if a server
// crashes.
func (g *RolloutConcurrencyGuard) AcquireDeviceAssignment(
    ctx context.Context,
    deviceID string,
    rolloutID string,
) (bool, error) {
    key := fmt.Sprintf("device_assignment:%s", deviceID)
    // SETNX with 24h TTL
    acquired, err := g.cache.SetNX(ctx, key, rolloutID,
        24*time.Hour)
    if err != nil {
        return false, fmt.Errorf("acquire device assignment lock:
        %w", err)
    }

    if acquired {
        g.cache.Publish(ctx, "device.assigned", map[string]string{
            "device_id": deviceID,
            "rollout_id": rolloutID,
        })
    }
    return acquired, nil
}

// ReleaseDeviceAssignment removes the assignment lock. Called
// when a
// rollout is halted or a device update completes.
func (g *RolloutConcurrencyGuard) ReleaseDeviceAssignment(
    ctx context.Context,
    deviceID string,
    rolloutID string,
) error {
    key := fmt.Sprintf("device_assignment:%s", deviceID)
    // Only release if the lock owner matches (prevents releasing
    // another rollout's lock)
    currentOwner, err := g.cache.Get(ctx, key)
    if err != nil {
        return nil // Key doesn't exist, already released
    }
    if currentOwner == rolloutID {
        return g.cache.Delete(ctx, key)
    }
    return nil // Owned by another rollout, don't release
}

```

4.6 Rollout Progression Scheduler

The decision engine is invoked on a configurable interval by the background task system:

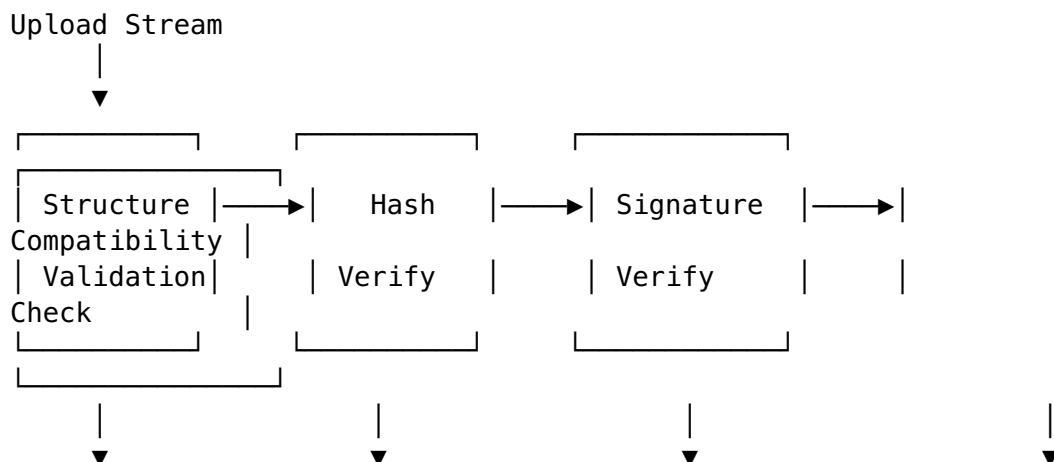
```
// ScheduleRolloutEvaluation registers the rollout evaluation task
// with the background task scheduler.
func ScheduleRolloutEvaluation(
    ctx context.Context,
    scheduler *backgroundtasks.Scheduler,
    engine *RolloutDecisionEngine,
    interval time.Duration,
) error {
    task := backgroundtasks.TaskConfig{
        Name:        "rollout_evaluation",
        Description:  "Evaluate all active rollouts and advance/
                    halt as needed",
        Interval:     interval,
        Timeout:      5 * time.Minute,
        MaxRetries:   3,
        RetryDelay:   30 * time.Second,
        Handler: func(ctx context.Context) error {
            return engine.EvaluateAll(ctx)
        },
    }

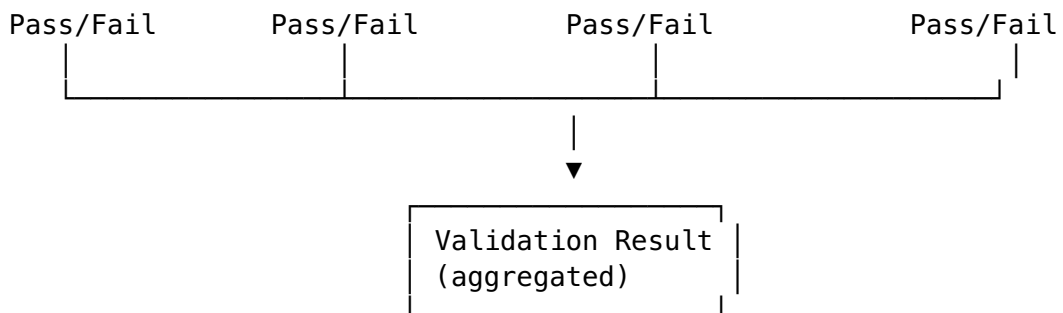
    return scheduler.Register(task)
}
```

5. Artifact Validation Pipeline

Artifact validation is a multi-stage pipeline that verifies update packages before they can be deployed to devices. Invalid artifacts must never reach devices — a corrupted firmware image could brick hardware.

5.1 Validation Pipeline Architecture





5.2 Complete Implementation

```
package validation
```

```
import (
    "context"
    "crypto"
    "crypto/rsa"
    "crypto/sha256"
    "encoding/hex"
    "fmt"
    "hash"
    "io"
    "sync"
    "time"

    "digital.vasic/concurrency"
    "digital.vasic/observability"

    "github.com/vasic-digital/helix-ota-server/internal/model"
)
```

```
// ValidationStage represents a single step in the validation pipeline.
```

```
type ValidationStage string
```

```
const (
    StageStructure      ValidationStage = "structure"
    StageHash           ValidationStage = "hash"
    StageSignature      ValidationStage = "signature"
    StageCompatibility ValidationStage = "compatibility"
)
```

```
// StageResult holds the outcome of a single validation stage.
```

```
type StageResult struct {
    Stage      ValidationStage `json:"stage"`
    Passed     bool               `json:"passed"`
    Message    string             `json:"message,omitempty"`
    Duration   time.Duration      `json:"duration"`
    Error      string             `json:"error,omitempty"`
    Details    map[string]string `json:"details,omitempty"`
}
```



```

// ValidationResult aggregates all stage results.
type ValidationResult struct {
    ArtifactID string      `json:"artifact_id"`
    Overall    bool                `json:"overall"`
    Stages     []StageResult    `json:"stages"`
    StartedAt  time.Time         `json:"started_at"`
    CompletedAt time.Time        `json:"completed_at"`
    TotalDuration time.Duration `json:"total_duration"`
}

// Validator is the interface for a single validation stage.
type Validator interface {
    Stage() ValidationStage
    Validate(ctx context.Context, artifact
        *model.Artifact, reader ArtifactReader) StageResult
}

// ArtifactReader provides streaming access to artifact data.
// Implementations may read from local disk, S3, or a network
stream.
type ArtifactReader interface {
    // NewReader returns a new io.ReadCloser for the artifact
    data.
    // Each call creates a fresh reader from the beginning.
    NewReader(ctx context.Context) (io.ReadCloser, error)
    // Size returns the total size of the artifact in bytes.
    Size() int64
}

// Pipeline orchestrates the sequential validation of an artifact
// through all registered validators.
type Pipeline struct {
    validators []Validator
    logger      *observability.Logger
    workerPool  *WorkerPool
}

// NewPipeline creates a validation pipeline with the standard
validators.
func NewPipeline(
    logger *observability.Logger,
    signingKey *rsa.PublicKey,
    hwCompatRepo HardwareCompatibilityRepository,
    poolSize int,
) *Pipeline {
    p := &Pipeline{
        logger:      logger,
        workerPool:  NewWorkerPool(poolSize),
    }

    // Register validators in execution order
    p.validators = []Validator{

```

```

        NewStructureValidator(logger),
        NewHashValidator(logger),
        NewSignatureValidator(logger, signingKey),
        NewCompatibilityValidator(logger, hwCompatRepo),
    }

    return p
}

// Validate executes all validation stages sequentially.
// A stage failure does not prevent subsequent stages from running
//
// we want a complete picture of all validation issues.
// The overall result is PASS only if ALL stages pass.
func (p *Pipeline) Validate(
    ctx context.Context,
    artifact *model.Artifact,
    reader ArtifactReader,
) *ValidationResult {
    result := &ValidationResult{
        ArtifactID: artifact.ID,
        Overall:     true,
        StartedAt:   time.Now().UTC(),
    }

    for _, validator := range p.validators {
        stageResult := validator.Validate(ctx, artifact, reader)
        result.Stages = append(result.Stages, stageResult)

        if !stageResult.Passed {
            result.Overall = false
            p.logger.Warn("validation stage failed",
                "artifact_id", artifact.ID,
                "stage", stageResult.Stage,
                "error", stageResult.Error,
            )
        }
    }

    result.CompletedAt = time.Now().UTC()
    result.TotalDuration =
        result.CompletedAt.Sub(result.StartedAt)
    return result
}

// ValidateBatch validates multiple artifacts concurrently using
// the worker pool.
func (p *Pipeline) ValidateBatch(
    ctx context.Context,
    artifacts []*model.Artifact,
    readers map[string]ArtifactReader,
) map[string]*ValidationResult {
    results := make(map[string]*ValidationResult, len(artifacts))

```

```

var mu sync.Mutex

g, ctx := concurrency.NewGroup(ctx,
    concurrency.WithMaxGoroutines(p.workerPool.size))

for _, artifact := range artifacts {
    artifact := artifact
    reader := readers[artifact.ID]

    g.Go(func() error {
        result := p.Validate(ctx, artifact, reader)
        mu.Lock()
        results[artifact.ID] = result
        mu.Unlock()
        return nil
    })
}

_ = g.Wait()
return results
}

```

5.3 Streaming Hash Validation

For large firmware images (up to 2 GB), we stream the hash computation to avoid loading the entire file into memory:

```

package validation

import (
    "context"
    "crypto/sha256"
    "encoding/hex"
    "fmt"
    "hash"
    "io"
    "time"

    "digital.vasic/observability"

    "github.com/vasic-digital/helix-ota-server/internal/model"
)

// HashValidator verifies artifact integrity using SHA-256 hash
// comparison.
// Uses streaming computation to handle large files without
// excessive memory use.
type HashValidator struct {
    logger *observability.Logger
}

func NewHashValidator(logger *observability.Logger)
    *HashValidator {

```

```

    return &HashValidator{logger: logger}
}

func (v *HashValidator) Stage() ValidationStage {
    return StageHash
}

func (v *HashValidator) Validate(
    ctx context.Context,
    artifact *model.Artifact,
    reader ArtifactReader,
) StageResult {
    start := time.Now()
    result := StageResult{
        Stage: StageHash,
        Details: map[string]string{
            "expected_sha256": artifact.SHA256Hash,
            "artifact_size":   fmt.Sprintf("%d bytes",
                reader.Size()),
        },
    }

    // Compute SHA-256 hash via streaming
    computedHash, err := v.computeHash(ctx, reader)
    if err != nil {
        result.Passed = false
        result.Error = fmt.Sprintf("hash computation failed: %v",
            err)
        result.Duration = time.Since(start)
        return result
    }

    result.Details["computed_sha256"] = computedHash

    if computedHash != artifact.SHA256Hash {
        result.Passed = false
        result.Error = fmt.Sprintf("hash mismatch: expected %s,
            got %s",
                artifact.SHA256Hash, computedHash)
        result.Message = "Artifact integrity check failed – file
            may be corrupted"
    } else {
        result.Passed = true
        result.Message = "SHA-256 hash verified successfully"
    }

    result.Duration = time.Since(start)
    return result
}

// computeHash streams the artifact data through SHA-256 in 64KB
// chunks.
// This uses a constant ~64KB of memory regardless of file size.

```

```

func (v *HashValidator) computeHash(
    ctx context.Context,
    reader ArtifactReader,
) (string, error) {
    r, err := reader.NewReader(ctx)
    if err != nil {
        return "", fmt.Errorf("create reader: %w", err)
    }
    defer r.Close()

    h := sha256.New()
    buf := make([]byte, 64*1024) // 64 KB buffer

    for {
        select {
        case <-ctx.Done():
            return "", ctx.Err()
        default:
        }

        n, readErr := io.ReadFull(r, buf)
        if n > 0 {
            h.Write(buf[:n])
        }
        if readErr == io.EOF || readErr == io.ErrUnexpectedEOF {
            break
        }
        if readErr != nil {
            return "", fmt.Errorf("read artifact data: %w",
                readErr)
        }
    }

    return hex.EncodeToString(h.Sum(nil)), nil
}

```

5.4 RSA Signature Verification

```
package validation
```

```

import (
    "context"
    "crypto"
    "crypto/rsa"
    "crypto/sha256"
    "crypto/x509"
    "encoding/pem"
    "fmt"
    "time"

    "digital.vasic/observability"

```

```

    "github.com/vasic-digital/helix-ota-server/internal/model"
)

// SignatureValidator verifies the RSA-PSS signature of an
// artifact.
// The signature is produced by the release engineering pipeline
// using
// the private key; this validator checks against the public key.
type SignatureValidator struct {
    publicKey *rsa.PublicKey
    logger    *observability.Logger
}

func NewSignatureValidator(
    logger *observability.Logger,
    publicKey *rsa.PublicKey,
) *SignatureValidator {
    return &SignatureValidator{
        publicKey: publicKey,
        logger:    logger,
    }
}

func (v *SignatureValidator) Stage() ValidationStage {
    return StageSignature
}

func (v *SignatureValidator) Validate(
    ctx context.Context,
    artifact *model.Artifact,
    reader ArtifactReader,
) StageResult {
    start := time.Now()
    result := StageResult{
        Stage: StageSignature,
    }

    if len(artifact.Signature) == 0 {
        result.Passed = false
        result.Error = "artifact has no signature"
        result.Message = "Unsigned artifact rejected – all
        artifacts must be signed"
        result.Duration = time.Since(start)
        return result
    }

    // Compute the SHA-256 digest of the artifact
    digest, err := v.computeDigest(ctx, reader)
    if err != nil {
        result.Passed = false
        result.Error = fmt.Sprintf("digest computation failed:
        %v", err)
        result.Duration = time.Since(start)
    }
}

```

```

        return result
    }

    // Verify RSA-PSS signature
    err = rsa.VerifyPSS(
        v.publicKey,
        crypto.SHA256,
        digest,
        artifact.Signature,
        &rsa.PSSOptions{
            SaltLength: rsa.PSSSaltLengthEqualsHash,
        },
    )

    if err != nil {
        result.Passed = false
        result.Error =
            fmt.Sprintf("signature verification failed: %v", err)
        result.Message = "Artifact signature is invalid –
            tampering detected"
        result.Duration = time.Since(start)
        return result
    }

    result.Passed = true
    result.Message = "RSA-PSS signature verified successfully"
    result.Details = map[string]string{
        "algorithm": "RSA-PSS-SHA256",
        "salt_length": "equals_hash",
        "signature_size": fmt.Sprintf("%d bytes",
            len(artifact.Signature)),
    }
    result.Duration = time.Since(start)
    return result
}

func (v *SignatureValidator) computeDigest(
    ctx context.Context,
    reader ArtifactReader,
) ([]byte, error) {
    r, err := reader.NewReader(ctx)
    if err != nil {
        return nil, fmt.Errorf("create reader: %w", err)
    }
    defer r.Close()

    h := sha256.New()
    buf := make([]byte, 64*1024)
    for {
        select {
        case <-ctx.Done():
            return nil, ctx.Err()
        default:

```

```

    }
    n, readErr := r.Read(buf)
    if n > 0 {
        h.Write(buf[:n])
    }
    if readErr == io.EOF {
        break
    }
    if readErr != nil {
        return nil, fmt.Errorf("read artifact: %w", readErr)
    }
}

return h.Sum(nil), nil
}

// ParseRSAPublicKeyFromPEM parses a PEM-encoded RSA public key.
func ParseRSAPublicKeyFromPEM(pemData []byte) (*rsa.PublicKey,
    error) {
    block, _ := pem.Decode(pemData)
    if block == nil {
        return nil, fmt.Errorf("failed to decode PEM block")
    }

    pub, err := x509.ParsePKIXPublicKey(block.Bytes)
    if err != nil {
        return nil, fmt.Errorf("parse public key: %w", err)
    }

    rsaPub, ok := pub.(*rsa.PublicKey)
    if !ok {
        return nil, fmt.Errorf("key is not RSA, got %T", pub)
    }

    return rsaPub, nil
}

```

5.5 Concurrent Validation Worker Pool

```

package validation

import (
    "context"
    "sync"

    "digital.vasic/concurrency"
)

// WorkerPool manages a pool of goroutines for concurrent artifact
// validation.
// Each worker picks validation jobs from a channel.
type WorkerPool struct {

```



```

    size      int
    jobs      chan validationJob
    results   chan *ValidationResult
    wg        sync.WaitGroup
}

type validationJob struct {
    artifact *model.Artifact
    reader   ArtifactReader
}

// NewWorkerPool creates a worker pool with the given concurrency.
func NewWorkerPool(size int) *WorkerPool {
    pool := &WorkerPool{
        size:      size,
        jobs:      make(chan validationJob, size*2),
        results:   make(chan *ValidationResult, size*2),
    }
    pool.start()
    return pool
}

func (p *WorkerPool) start() {
    for i := 0; i < p.size; i++ {
        p.wg.Add(1)
        go p.worker(i)
    }
}

func (p *WorkerPool) worker(id int) {
    defer p.wg.Done()
    for job := range p.jobs {
        // Validation is handled by the Pipeline, which the worker
        invokes
        _ = job // Actual validation logic called via
        Pipeline.Validate
    }
}

// Submit enqueues a validation job. Blocks if the pool is at
capacity.
func (p *WorkerPool) Submit(ctx context.Context, artifact
    *model.Artifact, reader ArtifactReader) error {
    select {
    case p.jobs <- validationJob{artifact: artifact, reader:
        reader}:
        return nil
    case <-ctx.Done():
        return ctx.Err()
    }
}

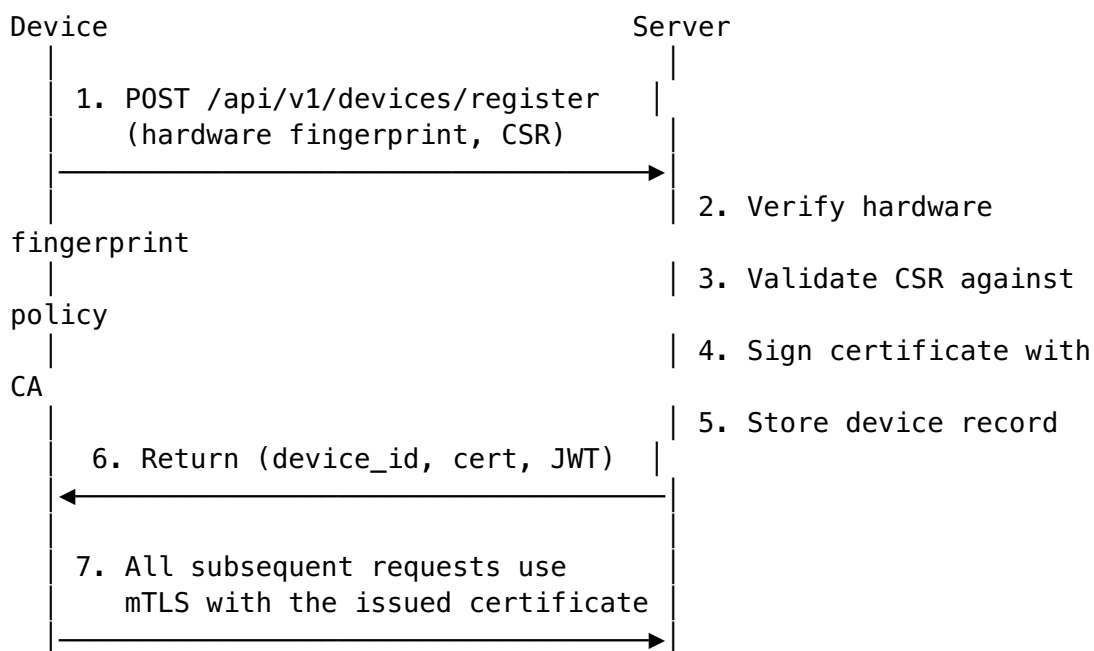
// Shutdown gracefully stops all workers after draining in-
progress jobs.

```

```
func (p *WorkerPool) Shutdown() {
    close(p.jobs)
    p.wg.Wait()
    close(p.results)
}
```

6. Device Registration & Authentication

6.1 mTLS Certificate Enrollment Flow



6.2 Device Identity from Hardware Properties

```
package auth
```

```
import (
    "crypto/sha256"
    "encoding/hex"
    "fmt"
    "sort"
)

// HardwareFingerprint represents the unique hardware properties
// of a device.
// These are collected by the client SDK and sent during
// registration.
type HardwareFingerprint struct {
    CPUID          string `json:"cpu_id"`
    BoardSerial    string `json:"board_serial"`
    MACAddress     string `json:"mac_address"`
    StorageSerial  string `json:"storage_serial"`
    TPMEndorsement string `json:"tpm_endorsement_key,omitempty"`
}
```

```

}

// GenerateDeviceID creates a deterministic, unique device
// identifier
// from the hardware fingerprint. The algorithm:
// 1. Sort the fingerprint fields by key name.
// 2. Concatenate key=value pairs with a separator.
// 3. Compute SHA-256 of the concatenation.
// 4. Prefix with "helix-" for namespace separation.
//
// This produces the SAME device ID for the same hardware, even
// across
// re-registrations (e.g., after a factory reset that preserves
// hardware).
func GenerateDeviceID(fp HardwareFingerprint) string {
    // Collect all non-empty fields
    fields := map[string]string{
        "cpu_id":      fp.CPUID,
        "board_serial": fp.BoardSerial,
        "mac_address":  fp.MACAddress,
        "storage_serial": fp.StorageSerial,
    }
    if fp.TPMEndorsement != "" {
        fields["tpm_endorsement"] = fp.TPMEndorsement
    }

    // Sort keys for deterministic ordering
    keys := make([]string, 0, len(fields))
    for k := range fields {
        keys = append(keys, k)
    }
    sort.Strings(keys)

    // Build concatenation
    concat := ""
    for _, k := range keys {
        if fields[k] != "" {
            concat += fmt.Sprintf("%s=%s|", k, fields[k])
        }
    }

    h := sha256.Sum256([]byte(concat))
    return "helix-" + hex.EncodeToString(h[:])[0:16]
}

```

6.3 JWT Token Management

```
package auth
```

```
import (
    "context"
    "fmt"
    "time"

```

```

    "digital.vasic/auth"
    "digital.vasic/cache"

    "github.com/vasic-digital/helix-ota-server/pkg/api"
)

// TokenManager handles JWT token lifecycle including issuance,
// validation, refresh, and revocation.
type TokenManager struct {
    jwtProvider *auth.JWTProvider
    tokenCache  cache.DistributedCache
    accessTTL   time.Duration
    refreshTTL  time.Duration
}

// NewTokenManager creates a new token manager with the given
// configuration.
func NewTokenManager(
    jwtProvider *auth.JWTProvider,
    tokenCache  cache.DistributedCache,
    accessTTL   time.Duration,
    refreshTTL  time.Duration,
) *TokenManager {
    return &TokenManager{
        jwtProvider: jwtProvider,
        tokenCache:  tokenCache,
        accessTTL:   accessTTL,
        refreshTTL:  refreshTTL,
    }
}

// IssueTokenPair generates an access/refresh token pair for a
// device.
func (tm *TokenManager) IssueTokenPair(
    ctx context.Context,
    deviceID string,
    roles []string,
) (*api.TokenPair, error) {
    now := time.Now().UTC()

    // Access token: short-lived, used for API calls
    accessToken, err := tm.jwtProvider.Generate(ctx,
        auth.TokenConfig{
            Subject:  deviceID,
            Issuer:   "helix-ota-server",
            IssuedAt: now,
            Expiry:   now.Add(tm.accessTTL),
            Claims:  map[string]interface{}{
                "roles":  roles,
                "type":   "access",
                "scope":  "device:read,update:check,telemetry:report",
            },
        },
    ),

```

```

    })
    if err != nil {
        return nil, fmt.Errorf("generate access token: %w", err)
    }

    // Refresh token: longer-lived, used only to obtain new access
    // tokens
    refreshToken, err := tm.jwtProvider.Generate(ctx,
        auth.TokenConfig{
            Subject:   deviceID,
            Issuer:    "helix-ota-server",
            IssuedAt:  now,
            Expiry:    now.Add(tm.refreshTTL),
            Claims:   map[string]interface{}{
                "type": "refresh",
            },
        })
    if err != nil {
        return nil, fmt.Errorf("generate refresh token: %w", err)
    }

    // Store refresh token in cache for validation and rotation
    // tracking
    refreshTokenKey := fmt.Sprintf("refresh_token:%s:%s", deviceID,
        refreshToken)
    err = tm.tokenCache.Set(ctx, refreshTokenKey, "valid",
        tm.refreshTTL)
    if err != nil {
        return nil, fmt.Errorf("store refresh token: %w", err)
    }

    return &api.TokenPair{
        AccessToken:  accessToken,
        RefreshToken: refreshToken,
        ExpiresIn:    int64(tm.accessTTL.Seconds()),
        TokenType:    "Bearer",
    }, nil
}

// ValidateAccessToken verifies an access token and returns its
// claims.
func (tm *TokenManager) ValidateAccessToken(
    ctx context.Context,
    tokenString string,
) (*api.TokenClaims, error) {
    // Check revocation list in cache
    revokedKey := fmt.Sprintf("revoked_token:%s", tokenString)
    exists, err := tm.tokenCache.Exists(ctx, revokedKey)
    if err != nil {
        return nil, fmt.Errorf("check revocation: %w", err)
    }
    if exists {
        return nil, fmt.Errorf("token has been revoked")
    }
}

```

```

// Validate JWT signature and claims
claims, err := tm.jwtProvider.Validate(ctx, tokenString)
if err != nil {
    return nil, fmt.Errorf("validate token: %w", err)
}

// Verify token type is "access"
if tokenType, ok := claims["type"].(string); !ok ||
    tokenType != "access" {
    return nil, fmt.Errorf("invalid token type: expected
    access")
}

return &api.TokenClaims{
    Subject:  claims.Subject(),
    Issuer:   claims.Issuer(),
    Roles:    claims["roles"].([]string),
    Scope:    claims["scope"].(string),
    IssuedAt: claims.IssuedAt().Unix(),
    Expiry:   claims.Expiry().Unix(),
}, nil
}

// RevokeToken adds a token to the revocation list until its
// natural expiry.
func (tm *TokenManager) RevokeToken(ctx context.Context,
    tokenString string) error {
    claims, err := tm.jwtProvider.Validate(ctx, tokenString)
    if err != nil {
        // Token is already invalid, nothing to revoke
        return nil
    }

    ttl := time.Until(claims.Expiry())
    if ttl <= 0 {
        return nil // Token already expired
    }

    revokedKey := fmt.Sprintf("revoked_token:%s", tokenString)
    return tm.tokenCache.Set(ctx, revokedKey, "revoked", ttl)
}

```

6.4 Device Heartbeat and Last-Seen Tracking

package service

```

import (
    "context"
    "fmt"
    "time"

    "digital.vasic/cache"

```

```

    "github.com/vasic-digital/helix-ota-server/internal/model"
)

// DeviceHeartbeatTracker manages device last-seen timestamps.
// Uses Redis for fast writes and periodically flushes to
// PostgreSQL
// for durability and analytics queries.
type DeviceHeartbeatTracker struct {
    deviceRepo DeviceRepository
    cache       cache.DistributedCache
}

// NewDeviceHeartbeatTracker creates a new heartbeat tracker.
func NewDeviceHeartbeatTracker(
    deviceRepo DeviceRepository,
    cache cache.DistributedCache,
) *DeviceHeartbeatTracker {
    return &DeviceHeartbeatTracker{
        deviceRepo: deviceRepo,
        cache:      cache,
    }
}

// RecordHeartbeat updates the last-seen timestamp for a device.
// Writes to Redis immediately (O(1)), PostgreSQL is updated
// asynchronously
// by the heartbeat flush background job.
func (t *DeviceHeartbeatTracker) RecordHeartbeat(
    ctx context.Context,
    deviceID string,
) error {
    key := fmt.Sprintf("heartbeat:%s", deviceID)
    now := time.Now().UTC().Unix()
    return t.cache.Set(ctx, key, now,
        0) // No TTL – cleaned up by flush job
}

// IsDeviceOnline checks if a device has been seen within the
// given threshold.
func (t *DeviceHeartbeatTracker) IsDeviceOnline(
    ctx context.Context,
    deviceID string,
    threshold time.Duration,
) (bool, error) {
    key := fmt.Sprintf("heartbeat:%s", deviceID)
    lastSeenUnix, err := t.cache.GetInt64(ctx, key)
    if err != nil {
        // Device has never sent a heartbeat or key expired
        return false, nil
    }

    lastSeen := time.Unix(lastSeenUnix, 0)

```

```

    return time.Since(lastSeen) <= threshold, nil
}

// FlushHeartbeatsToDB batch-writes cached heartbeat timestamps to
// PostgreSQL.
// Called periodically by a background job.
func (t *DeviceHeartbeatTracker) FlushHeartbeatsToDB(ctx
context.Context) (int, error) {
    // Scan all heartbeat keys from Redis
    keys, err := t.cache.ScanKeys(ctx, "heartbeat:*")
    if err != nil {
        return 0, fmt.Errorf("scan heartbeat keys: %w", err)
    }

    if len(keys) == 0 {
        return 0, nil
    }

    var updated int
    for _, key := range keys {
        deviceID := key[len("heartbeat:")]

        lastSeenUnix, err := t.cache.GetInt64(ctx, key)
        if err != nil {
            continue
        }

        lastSeen := time.Unix(lastSeenUnix, 0)
        if err := t.deviceRepo.UpdateLastSeen(ctx, deviceID,
lastSeen); err != nil {
            continue
        }
        updated++
    }

    return updated, nil
}

```

7. Telemetry Collection & Analysis

7.1 Event Ingestion Pipeline

```

package service

import (
    "context"
    "fmt"
    "time"

    "digital.vasic/eventbus"

```



```

    "digital.vasic/observability"

    "github.com/vasic-digital/helix-ota-server/internal/model"
    "github.com/vasic-digital/helix-ota-server/pkg/api"
    "github.com/vasic-digital/helix-ota-server/pkg/telemetry"
)

// TelemetryIngester receives device telemetry events, validates
// them,
// and routes them to storage and the event bus for real-time
// processing.
type TelemetryIngester struct {
    telemetryRepo TelemetryRepository
    eventBus      *eventbus.EventBus
    anomalyDetector *AnomalyDetector
    logger        *observability.Logger
    metrics       *telemetryIngestMetrics
}

type telemetryIngestMetrics struct {
    eventsIngested *observability.Counter
    eventsRejected *observability.Counter
    ingestionLatency *observability.Histogram
}

// IngestEvent processes a single telemetry event from a device.
func (ing *TelemetryIngester) IngestEvent(
    ctx context.Context,
    req *api.TelemetryEventRequest,
) error {
    start := time.Now()
    defer func() {
        ing.metrics.ingestionLatency.Observe(time.Since(start).Seconds())
    }()

    // Validate the event
    if err := ing.validateEvent(req); err != nil {
        ing.metrics.eventsRejected.Inc()
        return fmt.Errorf("validate telemetry event: %w", err)
    }

    // Enrich with server-side metadata
    event := &model.TelemetryEvent{
        ID: req.EventID,
        DeviceID: req.DeviceID,
        RolloutID: req.RolloutID,
        UpdateID: req.UpdateID,
        EventType: req.EventType,
        Timestamp: time.Now().UTC(), // Server-side
        timestamp
        ClientTimestamp: req.Timestamp,
        Payload: req.Payload,
        ServerRegion: req.ServerRegion,
    }

```

```

    }

    // Persist to database
    if err := ing.telemetryRepo.StoreEvent(ctx, event); err !=
        nil {
        return fmt.Errorf("store telemetry event: %w", err)
    }

    // Publish to event bus for real-time consumers (anomaly
    // detector, dashboards)
    ing.eventBus.Publish(ctx, eventbus.Event{
        Topic:  "telemetry.event",
        Key:    event.DeviceID,
        Payload: event,
    })

    ing.metrics.eventsIngested.Inc()
    return nil
}

// IngestBatch processes multiple telemetry events efficiently.
func (ing *TelemetryIngester) IngestBatch(
    ctx context.Context,
    events []*api.TelemetryEventRequest,
) *api.BatchIngestResult {
    result := &api.BatchIngestResult{
        Total:    len(events),
        Success:  0,
        Failed:   0,
        Errors:   make([]api.BatchError, 0),
    }

    for i, event := range events {
        if err := ing.IngestEvent(ctx, event); err != nil {
            result.Failed++
            result.Errors = append(result.Errors, api.BatchError{
                Index:    i,
                EventID:  event.EventID,
                Error:    err.Error(),
            })
        } else {
            result.Success++
        }
    }

    return result
}

func (ing *TelemetryIngester) validateEvent(req
    *api.TelemetryEventRequest) error {
    if req.DeviceID == "" {
        return fmt.Errorf("device_id is required")
    }
}

```

```

    if req.EventType == "" {
        return fmt.Errorf("event_type is required")
    }
    validTypes := map[string]bool{
        "download_started": true,
        "download_completed": true,
        "download_failed": true,
        "install_started": true,
        "install_completed": true,
        "install_failed": true,
        "rollback_started": true,
        "rollback_completed": true,
        "heartbeat": true,
    }
    if !validTypes[req.EventType] {
        return fmt.Errorf("invalid event_type: %s", req.EventType)
    }
    return nil
}

```

7.2 Real-Time Metrics Computation

```

package service

import (
    "context"
    "fmt"
    "time"

    "github.com/vasic-digital/helix-ota-server/pkg/api"
    "github.com/vasic-digital/helix-ota-server/pkg/telemetry"
)

// MetricsComputer calculates real-time aggregated metrics from
// telemetry events.
type MetricsComputer struct {
    telemetryRepo TelemetryRepository
}

// ComputeRolloutMetrics calculates key performance indicators for
// a rollout.
func (mc *MetricsComputer) ComputeRolloutMetrics(
    ctx context.Context,
    rolloutID string,
    window time.Duration,
) (*api.RolloutMetrics, error) {
    since := time.Now().UTC().Add(-window)

    stats, err := mc.telemetryRepo.GetRolloutStats(ctx,
        rolloutID, &since)
    if err != nil {
        return nil, fmt.Errorf("get rollout stats: %w", err)
    }
}

```

```

total := stats.CompletedCount + stats.FailedCount
var successRate, failureRate float64
if total > 0 {
    successRate = float64(stats.CompletedCount) /
        float64(total)
    failureRate = float64(stats.FailedCount) / float64(total)
}

return &api.RolloutMetrics{
    RolloutID:      rolloutID,
    Window:         window,
    TotalEvents:    stats.TotalEvents,
    DevicesCompleted: stats.CompletedCount,
    DevicesFailed:  stats.FailedCount,
    DevicesInProgress: stats.InProgressCount,
    SuccessRate:    successRate,
    FailureRate:    failureRate,
    AvgDownloadSpeed: stats.AvgDownloadSpeed,
    AvgInstallTime:  stats.AvgInstallDuration,
    ComputedAt:      time.Now().UTC(),
}, nil
}

```

7.3 Anomaly Detection

```

package service

import (
    "context"
    "fmt"
    "math"
    "sync"
    "time"

    "digital.vasic/eventbus"
    "digital.vasic/observability"

    "github.com/vasic-digital/helix-ota-server/internal/model"
    "github.com/vasic-digital/helix-ota-server/pkg/telemetry"
)

// AnomalyDetector monitors telemetry streams for abnormal
// patterns.
// It uses a sliding window algorithm to detect sudden spikes in
// failure rates.
type AnomalyDetector struct {
    telemetryRepo TelemetryRepository
    eventBus      *eventbus.EventBus
    logger        *observability.Logger

    mu sync.RWMutex
}

```

```

    baselines      map[string]*FailureBaseline // keyed by
        rollout_id
    alertCooldowns map[string]time.Time        // prevent alert
        storms

    // Configuration
    spikeThreshold float64 // e.g., 3.0 = 3x the baseline
        failure rate
    windowSize      time.Duration // sliding window for current
        rate
    baselineWindow  time.Duration // window for computing baseline
    cooldownPeriod  time.Duration // minimum time between alerts
        for same rollout
}

// FailureBaseline stores the expected failure rate for a rollout.
type FailureBaseline struct {
    RolloutID      string
    BaselineRate    float64
    CurrentRate     float64
    LastComputed    time.Time
}

// NewAnomalyDetector creates an anomaly detector with the given
// configuration.
func NewAnomalyDetector(
    telemetryRepo TelemetryRepository,
    eventBus *eventbus.EventBus,
    logger *observability.Logger,
    spikeThreshold float64,
) *AnomalyDetector {
    return &AnomalyDetector{
        telemetryRepo: telemetryRepo,
        eventBus:      eventBus,
        logger:        logger,
        baselines:     make(map[string]*FailureBaseline),
        alertCooldowns: make(map[string]time.Time),
        spikeThreshold: spikeThreshold,
        windowSize:     5 * time.Minute,
        baselineWindow: 1 * time.Hour,
        cooldownPeriod: 15 * time.Minute,
    }
}

// OnEvent is called by the event bus when a new telemetry event
// arrives.
// This is the real-time processing path – it must be fast.
func (d *AnomalyDetector) OnEvent(ctx context.Context, event
    interface{}) {
    te, ok := event.(*model.TelemetryEvent)
    if !ok {
        return
    }

```

```

// Only process failure events
if te.EventType != "install_failed" && te.EventType !=
    "download_failed" {
    return
}

if te.RolloutID == "" {
    return
}

d.evaluateAnomaly(ctx, te.RolloutID)
}

// evaluateAnomaly checks if the current failure rate for a
// rollout
// deviates significantly from the baseline.
func (d *AnomalyDetector) evaluateAnomaly(ctx context.Context,
    rolloutID string) {
    d.mu.Lock()
    defer d.mu.Unlock()

    // Check cooldown
    if lastAlert, exists := d.alertCooldowns[rolloutID]; exists {
        if time.Since(lastAlert) < d.cooldownPeriod {
            return // Skip - still in cooldown
        }
    }

    // Compute current failure rate (short window)
    since := time.Now().UTC().Add(-d.windowSize)
    currentStats, err := d.telemetryRepo.GetRolloutStats(ctx,
        rolloutID, &since)
    if err != nil {
        d.logger.Error("failed to compute current failure rate",
            "error", err)
        return
    }

    total := currentStats.CompletedCount +
        currentStats.FailedCount
    if total < 5 {
        return // Not enough data to make a determination
    }

    currentRate := float64(currentStats.FailedCount) /
        float64(total)

    // Get or compute baseline
    baseline, exists := d.baselines[rolloutID]
    if !exists || time.Since(baseline.LastComputed) >
        30*time.Minute {
        baselineSince := time.Now().UTC().Add(-d.baselineWindow)

```

```

baselineStats, err :=
d.telemetryRepo.GetRolloutStats(ctx, rolloutID,
&baselineSince)
if err != nil {
    return
}
baselineTotal := baselineStats.CompletedCount +
baselineStats.FailedCount
if baselineTotal == 0 {
    return
}
baseline = &FailureBaseline{
    RolloutID:    rolloutID,
    BaselineRate: float64(baselineStats.FailedCount) /
float64(baselineTotal),
    LastComputed: time.Now().UTC(),
}
d.baselines[rolloutID] = baseline
}

baseline.CurrentRate = currentRate

// Check for spike: is current rate significantly above
// baseline?
if baseline.BaselineRate > 0 && currentRate >
baseline.BaselineRate*d.spikeThreshold {
d.logger.Warn("ANOMALY DETECTED: failure rate spike",
    "rollout_id", rolloutID,
    "baseline_rate", fmt.Sprintf("%.2f%%",
baseline.BaselineRate*100),
    "current_rate", fmt.Sprintf("%.2f%%",
currentRate*100),
    "spike_factor", fmt.Sprintf("%.1fx", currentRate/
baseline.BaselineRate),
)

// Publish anomaly event
d.eventBus.Publish(ctx, eventbus.Event{
    Topic: "telemetry.anomaly",
    Key:    rolloutID,
    Payload: telemetry.AnomalyAlert{
        RolloutID:    rolloutID,
        AnomalyType:   "failure_rate_spike",
        BaselineRate: baseline.BaselineRate,
        CurrentRate:  currentRate,
        SpikeFactor:   currentRate /
baseline.BaselineRate,
        DetectedAt:    time.Now().UTC(),
        Severity:      d.computeSeverity(currentRate,
baseline.BaselineRate),
    },
})

d.alertCooldowns[rolloutID] = time.Now().UTC()

```

```

    }
}

// computeSeverity determines the severity level of an anomaly.
func (d *AnomalyDetector) computeSeverity(currentRate,
    baselineRate float64) telemetry.Severity {
    factor := currentRate / baselineRate
    switch {
    case factor >= 10.0 || currentRate > 0.5:
        return telemetry.SeverityCritical
    case factor >= 5.0 || currentRate > 0.3:
        return telemetry.SeverityHigh
    case factor >= 3.0 || currentRate > 0.15:
        return telemetry.SeverityMedium
    default:
        return telemetry.SeverityLow
    }
}

```

7.4 Time-Series Data Storage

```

// TelemetryRepository defines the data access interface for
telemetry events.
// Implementations should optimize for high-write throughput and
time-range queries.
type TelemetryRepository interface {
    // StoreEvent persists a single telemetry event.
    StoreEvent(ctx context.Context, event *model.TelemetryEvent)
        error

    // StoreBatch persists multiple events in a single
transaction.
    StoreBatch(ctx context.Context, events
        []*model.TelemetryEvent) error

    // GetRolloutStats computes aggregated statistics for a
rollout
// within the given time window.
    GetRolloutStats(ctx context.Context, rolloutID string, since
        *time.Time) (*RolloutStats, error)

    // QueryEvents retrieves raw events with time-range and type
filtering.
    QueryEvents(ctx context.Context, filter
        *TelemetryQueryFilter) ([]*model.TelemetryEvent, error)

    // GetTimeSeries returns bucketed metrics for time-series
visualization.
// Buckets are 1-minute, 5-minute, or 1-hour intervals.
    GetTimeSeries(ctx context.Context, req *TimeSeriesRequest)
        (*TimeSeriesResult, error)
}

// RolloutStats holds aggregated telemetry statistics.

```



```

type RolloutStats struct {
    TotalEvents      int
    CompletedCount   int
    FailedCount      int
    InProgressCount  int
    AvgDownloadDuration time.Duration
    AvgInstallDuration time.Duration
    AvgDownloadSpeed  float64 // bytes per second
}

func (s *RolloutStats) FailureRate() float64 {
    total := s.CompletedCount + s.FailedCount
    if total == 0 {
        return 0
    }
    return float64(s.FailedCount) / float64(total)
}

func (s *RolloutStats) SuccessRate() float64 {
    return 1.0 - s.FailureRate()
}

// TimeSeriesRequest specifies parameters for a time-series query.
type TimeSeriesRequest struct {
    RolloutID string
    Metric     string // "failure_rate", "success_rate",
        "download_speed"
    StartTime time.Time
    EndTime   time.Time
    BucketSize string // "1m", "5m", "1h"
}

// TimeSeriesResult contains bucketed metric values.
type TimeSeriesResult struct {
    Metric     string
    BucketSize string
    Buckets    []TimeBucket
}

type TimeBucket struct {
    Timestamp time.Time
    Value     float64
    Count     int // Number of events in this bucket
}

```

8. Background Jobs

The server runs several background jobs to maintain system health and progress rollouts. All jobs use the `digital.vasic/backgroundtasks` module for scheduling, retry, and monitoring.

8.1 Job Registry

```
package service
```

```
import (
    "context"
    "time"

    "digital.vasic/backgroundtasks"
    "digital.vasic/observability"
)

// RegisterBackgroundJobs sets up all scheduled background tasks.
func RegisterBackgroundJobs(
    ctx context.Context,
    scheduler *backgroundtasks.Scheduler,
    deps *BackgroundJobDependencies,
    logger *observability.Logger,
) error {
    jobs := []backgroundtasks.TaskConfig{
        {
            Name:        "rollout_evaluation",
            Description: "Evaluate active rollouts and advance/
halt as needed",
            Interval:    1 * time.Minute,
            Timeout:     5 * time.Minute,
            MaxRetries:  3,
            RetryDelay:  30 * time.Second,
            Handler: func(ctx context.Context) error {
                return deps.RolloutEngine.EvaluateAll(ctx)
            },
        },
        {
            Name:        "heartbeat_flush",
            Description: "Flush cached heartbeat timestamps to
PostgreSQL",
            Interval:    5 * time.Minute,
            Timeout:     2 * time.Minute,
            MaxRetries:  2,
            RetryDelay:  1 * time.Minute,
            Handler: func(ctx context.Context) error {
                _, err :=
deps.HeartbeatTracker.FlushHeartbeatsToDB(ctx)
                return err
            },
        },
        {
            Name:        "stale_device_cleanup",
            Description: "Mark devices as stale if not seen for 7
days",
            Interval:    1 * time.Hour,
            Timeout:     10 * time.Minute,
            MaxRetries:  2,
        },
    }
}
```

```

        RetryDelay: 5 * time.Minute,
        Handler: func(ctx context.Context) error {
            return
        }
    },
    deps.DeviceService.CleanupStaleDevices(ctx,
    7*24*time.Hour)
    },
    {
        Name: "artifact_cleanup",
        Description: "Remove expired artifacts from storage
and database",
        Interval: 6 * time.Hour,
        Timeout: 30 * time.Minute,
        MaxRetries: 2,
        RetryDelay: 10 * time.Minute,
        Handler: func(ctx context.Context) error {
            return deps.ArtifactService.CleanupExpired(ctx,
    30*24*time.Hour)
        },
    },
    {
        Name: "telemetry_aggregation",
        Description: "Compute and cache aggregated telemetry
metrics",
        Interval: 15 * time.Minute,
        Timeout: 5 * time.Minute,
        MaxRetries: 3,
        RetryDelay: 2 * time.Minute,
        Handler: func(ctx context.Context) error {
            return deps.TelemetryService.AggregateMetrics(ctx)
        },
    },
    {
        Name: "anomaly_detection_sweep",
        Description: "Periodic full sweep for anomalies across all active
rollouts",
        Interval: 5 * time.Minute,
        Timeout: 3 * time.Minute,
        MaxRetries: 1,
        Handler: func(ctx context.Context) error {
            return deps.AnomalyDetector.SweepAll(ctx)
        },
    },
}

for _, job := range jobs {
    if err := scheduler.Register(job); err != nil {
        return fmt.Errorf("register job %q: %w", job.Name,
err)
    }
    logger.Info("registered background job",
        "name", job.Name,
        "interval", job.Interval,

```

```

    )
}

return nil
}

// BackgroundJobDependencies holds all dependencies needed by
background jobs.
type BackgroundJobDependencies struct {
    RolloutEngine      *RolloutDecisionEngine
    HeartbeatTracker   *DeviceHeartbeatTracker
    DeviceService       DeviceService
    ArtifactService     ArtifactService
    TelemetryService    TelemetryService
    AnomalyDetector     *AnomalyDetector
}

```

8.2 Stale Device Cleanup

```

// CleanupStaleDevices marks devices as STALE if they haven't been
seen
// within the given threshold. Stale devices are excluded from new
rollouts
// but retain their telemetry history.
func (s *DeviceServiceImpl) CleanupStaleDevices(
    ctx context.Context,
    staleThreshold time.Duration,
) error {
    cutoff := time.Now().UTC().Add(-staleThreshold)

    devices, err := s.deviceRepo.ListByLastSeenBefore(ctx,
        cutoff, 1000)
    if err != nil {
        return fmt.Errorf("list stale devices: %w", err)
    }

    if len(devices) == 0 {
        return nil
    }

    var errors []error
    for _, device := range devices {
        device.Status = model.DeviceStatusStale
        device.UpdatedAt = time.Now().UTC()
        if err := s.deviceRepo.Update(ctx, device); err != nil {
            errors = append(errors, fmt.Errorf("mark device %s
                stale: %w", device.ID, err))
        }
    }

    if len(errors) > 0 {
        return fmt.Errorf("stale device cleanup: %d errors: %v",
            len(errors), errors[0])
    }
}

```

```

    }

    s.logger.Info("stale device cleanup completed",
        "marked_stale", len(devices),
        "threshold", staleThreshold,
    )
    return nil
}

```

9. Middleware Chain

The middleware chain is constructed using the `digital.vasic/middleware` module and follows a strict ordering to ensure correct behavior. The order matters: outer middleware runs first on the request path and last on the response path.

9.1 Middleware Order

Request →

1. Request ID (assign unique ID for tracing)
2. CORS (handle preflight, set headers)
3. Rate Limiter (reject excessive requests early)
4. Request Logging (log method, path, start time)
5. Recovery (catch panics, return 500)
6. Authentication (extract and validate JWT/mTLS)
7. Authorization (check RBAC permissions)
- Handler

Response ←

7. Authorization
6. Authentication
5. Recovery
4. Request Logging (log status, duration)
3. Rate Limiter
2. CORS
1. Request ID

9.2 Middleware Chain Implementation

```
package middleware
```

```
import (
    "net/http"
    "time"

    "digital.vasic/middleware"
    "digital.vasic/middleware/cors"
    "digital.vasic/middleware/ratelimiter"
    "digital.vasic/observability"
    "digital.vasic/recovery"

    "github.com/go-chi/chi/v5"

```

```

chimid "github.com/go-chi/chi/v5/middleware"

"github.com/vasic-digital/helix-ota-server/pkg/auth"
)

// ChainConfig holds configuration for the middleware chain.
type ChainConfig struct {
    // CORS configuration
    AllowedOrigins []string
    AllowedMethods []string
    AllowedHeaders []string
    AllowCredentials bool
    MaxAge          int

    // Rate limiting
    RateLimitRPS    float64 // Requests per second per IP
    RateLimitBurst  int     // Burst capacity
    RateLimitTTL    time.Duration // Window duration

    // Authentication
    TokenManager *auth.TokenManager
    PublicPaths  []string // Paths that skip authentication

    // Logging
    Logger *observability.Logger
}

// BuildChain constructs the complete middleware chain.
func BuildChain(config *ChainConfig) []func(http.Handler)
    http.Handler {
    var chain []func(http.Handler) http.Handler

    // 1. Request ID – assign a unique ID to every request for
    //    distributed tracing
    chain = append(chain, chimid.RequestID)

    // 2. CORS – handle cross-origin requests
    chain = append(chain, cors.Handler(cors.Config{
        AllowedOrigins:  config.AllowedOrigins,
        AllowedMethods:  config.AllowedMethods,
        AllowedHeaders:  config.AllowedHeaders,
        AllowCredentials: config.AllowCredentials,
        MaxAge:          config.MaxAge,
    })))

    // 3. Rate Limiter – using vasic-digital/ratelimiter
    limiterStore :=
        ratelimiter.NewRedisStore(nil) // Redis client injected
        separately
    chain = append(chain, ratelimiter.Handler(ratelimiter.Config{
        Rate:    config.RateLimitRPS,
        Burst:    config.RateLimitBurst,
        TTL:      config.RateLimitTTL,
    }))
}

```

```

        KeyFunc: ratelimiter.IPKeyFunc,
        Store:    limiterStore,
    )))

    // 4. Request Logging – log all requests with method, path,
        status, duration
    chain = append(chain, RequestLogger(config.Logger))

    // 5. Recovery – catch panics, log stack trace, return 500
    chain = append(chain, recovery.HTTPHandler(recovery.Config{
        LogStack: true,
        Logger:    config.Logger,
    })))

    // 6. Authentication – extract JWT or mTLS client certificate
    chain = append(chain, NewAuthMiddleware(config.TokenManager,
        config.PublicPaths))

    // 7. Authorization – RBAC check based on user/device roles
    chain = append(chain, NewRBACMiddleware())

    return chain
}

// RequestLogger returns middleware that logs each request.
func RequestLogger(logger observability.Logger)
    func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
        *http.Request) {
        start := time.Now()
        ww := chimid.NewWrapResponseWriter(w, r.ProtoMajor)

        defer func() {
            logger.Info("http request",
                "method", r.Method,
                "path", r.URL.Path,
                "status", ww.Status(),
                "bytes", ww.BytesWritten(),
                "duration_ms",
                time.Since(start).Milliseconds(),
                "request_id", chimid.GetReqID(r.Context()),
                "remote_addr", r.RemoteAddr,
            )
        }()

        next.ServeHTTP(ww, r)
    })
    }
}

```

9.3 Authentication Middleware

```
package middleware

import (
    "context"
    "net/http"
    "strings"

    "github.com/vasic-digital/helix-ota-server/pkg/api"
    "github.com/vasic-digital/helix-ota-server/pkg/auth"
)

// AuthMiddleware extracts and validates authentication
// credentials
// from incoming requests. Supports two modes:
// - Bearer token (JWT) in the Authorization header
// - mTLS client certificate (verified by the TLS terminator)
type AuthMiddleware struct {
    tokenManager *auth.TokenManager
    publicPaths  map[string]bool
}

func NewAuthMiddleware(
    tokenManager *auth.TokenManager,
    publicPaths []string,
) *AuthMiddleware {
    paths := make(map[string]bool, len(publicPaths))
    for _, p := range publicPaths {
        paths[p] = true
    }
    return &AuthMiddleware{
        tokenManager: tokenManager,
        publicPaths:  paths,
    }
}

func (m *AuthMiddleware) ServeHTTP(next http.Handler)
    http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
        *http.Request) {
        // Skip authentication for public paths
        if m.publicPaths[r.URL.Path] {
            next.ServeHTTP(w, r)
            return
        }

        // Try JWT authentication first
        claims, err := m.extractJWT(r)
        if err == nil && claims != nil {
            ctx := context.WithValue(r.Context(), authKey{},
                claims)
            next.ServeHTTP(w, r.WithContext(ctx))
        }
    })
}
```



```

        return
    }

    // Try mTLS authentication
    deviceID, err := m.extractMTLS(r)
    if err == nil && deviceID != "" {
        ctx := context.WithValue(r.Context(), deviceIDKey{},
            deviceID)
        next.ServeHTTP(w, r.WithContext(ctx))
        return
    }

    // No valid authentication
    http.Error(w, `{"error":"unauthorized","message":"valid
    authentication required"}`, http.StatusUnauthorized)
})
}

func (m *AuthMiddleware) extractJWT(r *http.Request)
    (*api.TokenClaims, error) {
    authHeader := r.Header.Get("Authorization")
    if authHeader == "" {
        return nil, fmt.Errorf("no authorization header")
    }

    parts := strings.SplitN(authHeader, " ", 2)
    if len(parts) != 2 || !strings.EqualFold(parts[0], "Bearer") {
        return nil, fmt.Errorf("invalid authorization header
        format")
    }

    return m.tokenManager.ValidateAccessToken(r.Context(),
        parts[1])
}

func (m *AuthMiddleware) extractMTLS(r *http.Request) (string,
    error) {
    if r.TLS == nil || len(r.TLS.PeerCertificates) == 0 {
        return "", fmt.Errorf("no client certificate")
    }

    cert := r.TLS.PeerCertificates[0]
    // Verify the certificate was issued by our CA
    // The subject Common Name contains the device ID
    deviceID := cert.Subject.CommonName
    if deviceID == "" {
        return "", fmt.Errorf("certificate has no device ID in
        CN")
    }

    return deviceID, nil
}

// Context key types to avoid collisions.

```

```

type authKey struct{}
type deviceIDKey struct{}

// GetTokenClaims extracts token claims from the request context.
func GetTokenClaims(ctx context.Context) *api.TokenClaims {
    claims, _ := ctx.Value(authKey{}).(*api.TokenClaims)
    return claims
}

// GetDeviceID extracts the mTLS device ID from the request
// context.
func GetDeviceID(ctx context.Context) string {
    id, _ := ctx.Value(deviceIDKey{}).(string)
    return id
}

```

9.4 RBAC Middleware

```

package middleware

import (
    "context"
    "net/http"
    "strings"

    "github.com/vasic-digital/helix-ota-server/pkg/api"
)

// Role constants
const (
    RoleAdmin      = "admin"
    RoleOperator   = "operator"
    RoleViewer     = "viewer"
    RoleDevice     = "device"
)

// Permission constants
const (
    PermRolloutCreate = "rollout:create"
    PermRolloutUpdate = "rollout:update"
    PermRolloutHalt   = "rollout:halt"
    PermDeviceRead    = "device:read"
    PermDeviceWrite   = "device:write"
    PermArtifactUpload = "artifact:upload"
    PermArtifactDelete = "artifact:delete"
    PermTelemetryRead  = "telemetry:read"
    PermUpdateCheck   = "update:check"
)

// rolePermissions maps roles to their allowed permissions.
var rolePermissions = map[string]map[string]bool{
    RoleAdmin: {

```

```

        PermRolloutCreate: true, PermRolloutUpdate: true,
        PermRolloutHalt: true,
        PermDeviceRead: true, PermDeviceWrite: true,
        PermArtifactUpload: true, PermArtifactDelete: true,
        PermTelemetryRead: true, PermUpdateCheck: true,
    },
    RoleOperator: {
        PermRolloutCreate: true, PermRolloutUpdate: true,
        PermRolloutHalt: true,
        PermDeviceRead: true, PermDeviceWrite: true,
        PermArtifactUpload: true, PermTelemetryRead: true,
    },
    RoleViewer: {
        PermRolloutUpdate: false, PermDeviceRead: true,
        PermTelemetryRead: true,
    },
    RoleDevice: {
        PermUpdateCheck: true,
    },
}

// RBACMiddleware checks role-based permissions for each request.
type RBACMiddleware struct {
    routePermissions
    map[string]string // route pattern → required permission
}

func NewRBACMiddleware() *RBACMiddleware {
    return &RBACMiddleware{
        routePermissions: map[string]string{
            "POST /api/v1/rollouts": PermRolloutCreate,
            "PUT /api/v1/rollouts/*": PermRolloutUpdate,
            "DELETE /api/v1/rollouts/*": PermRolloutHalt,
            "GET /api/v1/devices": PermDeviceRead,
            "POST /api/v1/devices": PermDeviceWrite,
            "PUT /api/v1/devices/*": PermDeviceWrite,
            "POST /api/v1/artifacts": PermArtifactUpload,
            "DELETE /api/v1/artifacts/*": PermArtifactDelete,
            "GET /api/v1/telemetry": PermTelemetryRead,
            "POST /api/v1/updates/check": PermUpdateCheck,
        },
    }
}

func (m *RBACMiddleware) ServeHTTP(next http.Handler)
    http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
        *http.Request) {
        routeKey := r.Method + " " + r.URL.Path
        requiredPerm, exists := m.routePermissions[routeKey]
        if !exists {
            // No permission requirement – allow all authenticated
            users
            next.ServeHTTP(w, r)

```

```

        return
    }

    // Extract roles from context
    var roles []string
    if claims := GetTokenClaims(r.Context()); claims != nil {
        roles = claims.Roles
    } else if deviceID := GetDeviceID(r.Context()); deviceID != "" {
        roles = []string{RoleDevice}
    }

    // Check if any role has the required permission
    if !m.hasPermission(roles, requiredPerm) {
        http.Error(w,
            `{"error":"forbidden","message":"insufficient permissions"}`,
            http.StatusForbidden)
        return
    }

    next.ServeHTTP(w, r)
})
}

func (m *RBACMiddleware) hasPermission(roles []string, permission string) bool {
    for _, role := range roles {
        if perms, ok := rolePermissions[role]; ok {
            if perms[permission] {
                return true
            }
        }
    }
    return false
}

```

10. Server Main Implementation

The main entry point wires all dependencies together, configures the HTTP server, and manages the application lifecycle including graceful shutdown.

```

package main

import (
    "context"
    "fmt"
    "net/http"
    "os"
    "os/signal"
    "syscall"

```

```

"time"

"digital.vasic/auth"
"digital.vasic/backgroundtasks"
"digital.vasic/cache"
"digital.vasic/config"
"digital.vasic/database"
"digital.vasic/eventbus"
"digital.vasic/middleware/cors"
"digital.vasic/observability"
"digital.vasic/ratelimiter"
"digital.vasic/recovery"
"digital.vasic/storage"

"github.com/go-chi/chi/v5"
chimid "github.com/go-chi/chi/v5/middleware"
"github.com/jackc/pgx/v5/pgxpool"

"github.com/vasic-digital/helix-ota-server/internal/handler"
"github.com/vasic-digital/helix-ota-server/internal/
middleware"
"github.com/vasic-digital/helix-ota-server/internal/
repository/postgres"
"github.com/vasic-digital/helix-ota-server/internal/
repository/redis"
"github.com/vasic-digital/helix-ota-server/internal/service"
"github.com/vasic-digital/helix-ota-server/internal/
validation"
pkgauth "github.com/vasic-digital/helix-ota-server/pkg/auth"
)

// @title Helix OTA Server API
// @version 1.0.0
// @description Enterprise-grade Over-The-Air update management
//      system

func main() {
    ctx := context.Background()

    // — 1. Load Configuration —
    cfg, err := loadConfig()
    if err != nil {
        fmt.Fprintf(os.Stderr, "failed to load config: %v\n", err)
        os.Exit(1)
    }

    // — 2. Initialize Logger —
    logger := observability.NewLogger(observability.Config{
        Level:  cfg.LogLevel,
        Format: cfg.LogFormat,
    })

    logger.Info("starting Helix OTA server",

```

```

        "version", "1.0.0-mvp",
        "environment", cfg.Environment,
    )

    // — 3. Connect to PostgreSQL —
    dbPool, err := database.Connect(ctx, database.Config{
        Host:      cfg.Database.Host,
        Port:      cfg.Database.Port,
        User:      cfg.Database.User,
        Password:  cfg.Database.Password,
        Database:  cfg.Database.Name,
        MaxConns:  cfg.Database.MaxConnections,
        MinConns:  cfg.Database.MinConnections,
    })
    if err != nil {
        logger.Error("failed to connect to database", "error",
            err)
        os.Exit(1)
    }
    defer dbPool.Close()

    logger.Info("connected to PostgreSQL",
        "host", cfg.Database.Host,
        "database", cfg.Database.Name,
    )

    // — 4. Connect to Redis —
    redisClient, err := cache.Connect(ctx, cache.Config{
        Addr:      fmt.Sprintf("%s:%d", cfg.Redis.Host,
            cfg.Redis.Port),
        Password:  cfg.Redis.Password,
        DB:        cfg.Redis.DB,
    })
    if err != nil {
        logger.Error("failed to connect to Redis", "error", err)
        os.Exit(1)
    }
    defer redisClient.Close()

    logger.Info("connected to Redis",
        "host", cfg.Redis.Host,
    )

    // — 5. Run Database Migrations —
    if cfg.RunMigrations {
        migrator := database.NewMigrator(dbPool, "migrations")
        if err := migrator.Up(ctx); err != nil {
            logger.Error("failed to run migrations", "error", err)
            os.Exit(1)
        }
        logger.Info("database migrations completed")
    }
}

```

```

// — 6. Initialize Object Storage —
storageClient, err := storage.Connect(ctx, storage.Config{
    Provider: cfg.Storage.Provider,
    Bucket:   cfg.Storage.Bucket,
    Region:   cfg.Storage.Region,
    Endpoint: cfg.Storage.Endpoint,
})
if err != nil {
    logger.Error("failed to connect to object storage",
        "error", err)
    os.Exit(1)
}

// — 7. Initialize Event Bus —
eventBus := eventbus.New(eventbus.Config{
    BufferSize: 1000,
    Workers:    cfg.EventBusWorkers,
})
defer eventBus.Close()

// — 8. Initialize JWT Provider —
jwtProvider, err := auth.NewJWTProvider(auth.JWTConfig{
    SigningKey:    cfg.Auth.JWTSigningKey,
    Issuer:         "helix-ota-server",
    AccessTTL:      cfg.Auth.AccessTokenTTL,
    RefreshTTL:     cfg.Auth.RefreshTokenTTL,
})
if err != nil {
    logger.Error("failed to initialize JWT provider",
        "error", err)
    os.Exit(1)
}

// — 9. Initialize Repositories —
repos := &RepositorySet{
    UpdateRepo:    postgres.NewUpdateRepository(dbPool),
    DeviceRepo:    postgres.NewDeviceRepository(dbPool),
    RolloutRepo:   postgres.NewRolloutRepository(dbPool),
    ArtifactRepo:  postgres.NewArtifactRepository(dbPool),
    TelemetryRepo: postgres.NewTelemetryRepository(dbPool),
    DeviceCache:   redis.NewDeviceCache(redisClient),
    RolloutCache:  redis.NewRolloutCache(redisClient),
}

// — 10. Initialize Services —
tokenManager := pkgauth.NewTokenManager(jwtProvider,
    redisClient,
    cfg.Auth.AccessTokenTTL, cfg.Auth.RefreshTokenTTL)

heartbeatTracker :=
    service.NewDeviceHeartbeatTracker(repos.DeviceRepo,
    redisClient)

```

```

rolloutEngine := service.NewRolloutDecisionEngine(
    repos.RolloutRepo, repos.DeviceRepo, repos.TelemetryRepo,
    eventBus, logger,
)

signingKey, err :=
    pkgauth.ParseRSAPublicKeyFromPEM(cfg.Artifact.SigningPublicKey)
if err != nil {
    logger.Error("failed to parse artifact signing key",
        "error", err)
    os.Exit(1)
}

validationPipeline := validation.NewPipeline(logger,
    signingKey, repos.ArtifactRepo, 4)

anomalyDetector := service.NewAnomalyDetector(
    repos.TelemetryRepo, eventBus, logger, 3.0,
)

services := &ServiceSet{
    UpdateService:
        service.NewUpdateService(repos.UpdateRepo,
            repos.RolloutRepo, rolloutEngine, logger),
    DeviceService:
        service.NewDeviceService(repos.DeviceRepo,
            repos.DeviceCache, logger),
    RolloutService:
        service.NewRolloutService(repos.RolloutRepo,
            rolloutEngine, eventBus, logger),
    ArtifactService:
        service.NewArtifactService(repos.ArtifactRepo,
            storageClient, validationPipeline, eventBus, logger),
    TelemetryService:
        service.NewTelemetryService(repos.TelemetryRepo,
            eventBus, anomalyDetector, logger),
    AuthService:
        service.NewAuthService(repos.DeviceRepo, tokenManager,
            jwtProvider, logger),
}

// — 11. Register Event Bus Subscribers —
eventBus.Subscribe("telemetry.event", anomalyDetector.OnEvent)

// — 12. Initialize Background Jobs —
taskScheduler :=
    backgroundtasks.NewScheduler(backgroundtasks.SchedulerConfig{
        Logger: logger,
    })

if err := service.RegisterBackgroundJobs(ctx, taskScheduler,
    &service.BackgroundJobDependencies{
        RolloutEngine: rolloutEngine,
        HeartbeatTracker: heartbeatTracker,
        DeviceService: services.DeviceService,
    },

```



```

    ArtifactService: services.ArtifactService,
    TelemetryService: services.TelemetryService,
    AnomalyDetector: anomalyDetector,
}, logger); err != nil {
    logger.Error("failed to register background jobs",
        "error", err)
    os.Exit(1)
}

taskScheduler.Start()
defer taskScheduler.Stop()

// — 13. Setup HTTP Router and Middleware —
router := chi.NewRouter()

// Apply middleware chain (outer to inner)
middlewareChain :=
    middleware.BuildChain(&middleware.ChainConfig{
        AllowedOrigins:    cfg.CORS.AllowedOrigins,
        AllowedMethods:    cfg.CORS.AllowedMethods,
        AllowedHeaders:    cfg.CORS.AllowedHeaders,
        AllowCredentials:  cfg.CORS.AllowCredentials,
        MaxAge:            cfg.CORS.MaxAge,
        RateLimitRPS:      cfg.RateLimit.RPS,
        RateLimitBurst:    cfg.RateLimit.Burst,
        RateLimitTTL:      cfg.RateLimit.TTL,
        TokenManager:      tokenManager,
        PublicPaths:       []string{"/api/v1/auth/login", "/api/v1/auth/register", "/healthz", "/readyz"},
        Logger:            logger,
    })

router.Use(middlewareChain...)

// — 14. Register Routes —
registerRoutes(router, services, logger)

// — 15. Start HTTP Server —
httpServer := &http.Server{
    Addr:    fmt.Sprintf(":%d", cfg.Server.Port),
    Handler: router,
    ReadTimeout:  cfg.Server.ReadTimeout,
    WriteTimeout: cfg.Server.WriteTimeout,
    IdleTimeout:  cfg.Server.IdleTimeout,
}

// Start server in a goroutine
go func() {
    logger.Info("HTTP server starting",
        "port", cfg.Server.Port,
    )
    if err := httpServer.ListenAndServe(); err != nil && err != http.ErrServerClosed {

```

```

        logger.Error("HTTP server error", "error", err)
        os.Exit(1)
    }
}()

// — 16. Graceful Shutdown —
quit := make(chan os.Signal, 1)
signal.Notify(qquit, syscall.SIGINT, syscall.SIGTERM)
sig := <-quit

logger.Info("shutdown signal received", "signal",
    sig.String())

shutdownCtx, shutdownCancel :=
    context.WithTimeout(context.Background(), 30*time.Second)
defer shutdownCancel()

// Stop accepting new requests
if err := httpServer.Shutdown(shutdownCtx); err != nil {
    logger.Error("HTTP server shutdown error", "error", err)
}

// Stop background tasks
taskScheduler.Stop()

// Close event bus (drains pending events)
eventBus.Close()

logger.Info("server shutdown complete")
}

// —————
// Configuration
// —————

type ServerConfig struct {
    Environment string
    Server      ServerSettings
    Database    DatabaseSettings
    Redis       RedisSettings
    Auth        AuthSettings
    CORS        COSSSettings
    RateLimit   RateLimitSettings
    Artifact    ArtifactSettings
    Storage     StorageSettings
    EventBusWorkers int
    LogLevel    string
    LogFormat   string
    RunMigrations bool
}

type ServerSettings struct {
    Port int

```

```
    ReadTimeout    time.Duration
    WriteTimeout    time.Duration
    IdleTimeout     time.Duration
}
```

```
type DatabaseSettings struct {
    Host          string
    Port          int
    User          string
    Password      string
    Name          string
    MaxConnections int32
    MinConnections int32
}
```

```
type RedisSettings struct {
    Host      string
    Port      int
    Password  string
    DB        int
}
```

```
type AuthSettings struct {
    JWTSigningKey []byte
    AccessTokenTTL time.Duration
    RefreshTokenTTL time.Duration
}
```

```
type COSSSettings struct {
    AllowedOrigins []string
    AllowedMethods []string
    AllowedHeaders []string
    AllowCredentials bool
    MaxAge          int
}
```

```
type RateLimitSettings struct {
    RPS    float64
    Burst  int
    TTL    time.Duration
}
```

```
type ArtifactSettings struct {
    SigningPublicKey []byte
    MaxUploadSize    int64
}
```

```
type StorageSettings struct {
    Provider string
    Bucket   string
    Region   string
    Endpoint string
}
```

```

func loadConfig() (*ServerConfig, error) {
    loader := config.NewLoader(config.ConfigOptions{
        EnvPrefix: "HELIX_OTA",
        Files:     []string{"config.yaml", "/etc/helix-ota/
        config.yaml"},
    })

    var cfg ServerConfig
    if err := loader.Load(&cfg); err != nil {
        return nil, fmt.Errorf("load configuration: %w", err)
    }

    // Apply defaults
    if cfg.Server.Port == 0 {
        cfg.Server.Port = 8080
    }
    if cfg.Server.ReadTimeout == 0 {
        cfg.Server.ReadTimeout = 15 * time.Second
    }
    if cfg.Server.WriteTimeout == 0 {
        cfg.Server.WriteTimeout = 60 * time.Second
    }
    if cfg.Server.IdleTimeout == 0 {
        cfg.Server.IdleTimeout = 120 * time.Second
    }
    if cfg.Auth.AccessTokenTTL == 0 {
        cfg.Auth.AccessTokenTTL = 15 * time.Minute
    }
    if cfg.Auth.RefreshTokenTTL == 0 {
        cfg.Auth.RefreshTokenTTL = 7 * 24 * time.Hour
    }
    if cfg.RateLimit.RPS == 0 {
        cfg.RateLimit.RPS = 100
    }
    if cfg.RateLimit.Burst == 0 {
        cfg.RateLimit.Burst = 200
    }
    if cfg.EventBusWorkers == 0 {
        cfg.EventBusWorkers = 4
    }

    return &cfg, nil
}

```

```

// Route Registration

```

```

func registerRoutes(
    router chi.Router,
    services *ServiceSet,
    logger *observability.Logger,

```

```

) {
    // Health endpoints (no auth required)
    router.Get("/healthz", handler.Healthz)
    router.Get("/readyz", handler.Readyz(services.DeviceService))

    // Authentication
    router.Route("/api/v1/auth", func(r chi.Router) {
        r.Post("/login", handler.Login(services.AuthService,
            logger))
        r.Post("/refresh",
            handler.RefreshToken(services.AuthService, logger))
        r.Post("/register",
            handler.RegisterDevice(services.AuthService, logger))
    })

    // Updates
    router.Route("/api/v1/updates", func(r chi.Router) {
        r.Post("/check",
            handler.CheckForUpdate(services.UpdateService, logger))
        r.Get("/{updateID}",
            handler.GetUpdate(services.UpdateService, logger))
        r.Get("/", handler.ListUpdates(services.UpdateService,
            logger))
    })

    // Devices
    router.Route("/api/v1/devices", func(r chi.Router) {
        r.Post("/",
            handler.RegisterDeviceHTTP(services.DeviceService,
            logger))
        r.Get("/", handler.ListDevices(services.DeviceService,
            logger))
        r.Get("/{deviceID}",
            handler.GetDevice(services.DeviceService, logger))
        r.Put("/{deviceID}",
            handler.UpdateDevice(services.DeviceService, logger))
        r.Delete("/{deviceID}",
            handler.DecommissionDevice(services.DeviceService,
            logger))
        r.Post("/{deviceID}/heartbeat",
            handler.DeviceHeartbeat(services.DeviceService, logger))
    })

    // Rollouts
    router.Route("/api/v1/rollouts", func(r chi.Router) {
        r.Post("/",
            handler.CreateRollout(services.RolloutService, logger))
        r.Get("/", handler.ListRollouts(services.RolloutService,
            logger))
        r.Get("/{rolloutID}",
            handler.GetRollout(services.RolloutService, logger))
        r.Put("/{rolloutID}",
            handler.UpdateRollout(services.RolloutService, logger))
        r.Post("/{rolloutID}/pause",
            handler.PauseRollout(services.RolloutService, logger))
    })
}

```

```

        r.Post("/{rolloutID}/resume",
            handler.ResumeRollout(services.RolloutService, logger))
        r.Post("/{rolloutID}/halt",
            handler.HaltRollout(services.RolloutService, logger))
        r.Get("/{rolloutID}/progress",
            handler.GetRolloutProgress(services.RolloutService,
            logger))
    })

    // Artifacts
    router.Route("/api/v1/artifacts", func(r chi.Router) {
        r.Post("/",
            handler.UploadArtifact(services.ArtifactService, logger))
        r.Get("/",
            handler.ListArtifacts(services.ArtifactService, logger))
        r.Get("/{artifactID}",
            handler.GetArtifact(services.ArtifactService, logger))
        r.Delete("/{artifactID}",
            handler.DeleteArtifact(services.ArtifactService, logger))
        r.Post("/{artifactID}/validate",
            handler.ValidateArtifact(services.ArtifactService,
            logger))
        r.Get("/{artifactID}/download",
            handler.DownloadArtifact(services.ArtifactService,
            logger))
    })

    // Telemetry
    router.Route("/api/v1/telemetry", func(r chi.Router) {
        r.Post("/events",
            handler.ReportTelemetryEvent(services.TelemetryService,
            logger))
        r.Post("/events/batch",
            handler.ReportTelemetryBatch(services.TelemetryService,
            logger))
        r.Get("/overview",
            handler.GetTelemetryOverview(services.TelemetryService,
            logger))
        r.Get("/devices/{deviceID}",
            handler.GetDeviceTelemetry(services.TelemetryService,
            logger))
    })
}

// -----
// Dependency Containers
// -----

type RepositorySet struct {
    UpdateRepo    postgres.UpdateRepository
    DeviceRepo    postgres.DeviceRepository
    RolloutRepo   postgres.RolloutRepository
    ArtifactRepo  postgres.ArtifactRepository
    TelemetryRepo postgres.TelemetryRepository
    DeviceCache   redis.DeviceCache

```

```

RolloutCache  redis.RolloutCache
}

type ServiceSet struct {
    UpdateService    service.UpdateService
    DeviceService    service.DeviceService
    RolloutService   service.RolloutService
    ArtifactService  service.ArtifactService
    TelemetryService service.TelemetryService
    AuthService      service.AuthService
}

```

11. Integration with vasic-digital Submodules

Each vasic-digital submodule provides a self-contained capability. Below is the import, initialization, and usage pattern for every submodule used by the Helix OTA server.

11.1 digital.vasic/auth

JWT generation, validation, and key management.

```

import "digital.vasic/auth"

// Initialization (in main.go)
jwtProvider, err := auth.NewJWTProvider(auth.JWTConfig{
    SigningKey:    cfg.Auth.JWTSigningKey,
    Issuer:        "helix-ota-server",
    AccessTTL:     cfg.Auth.AccessTokenTTL,
    RefreshTTL:    cfg.Auth.RefreshTokenTTL,
})
if err != nil {
    logger.Error("failed to initialize JWT provider", "error",
        err)
    os.Exit(1)
}

// Usage: Generate a token
token, err := jwtProvider.Generate(ctx, auth.TokenConfig{
    Subject: deviceID,
    Issuer:  "helix-ota-server",
    IssuedAt: time.Now().UTC(),
    Expiry:   time.Now().UTC().Add(15 * time.Minute),
    Claims: map[string]interface{}{
        "roles": []string{"device"},
        "type":  "access",
    },
})

// Usage: Validate a token

```

```
claims, err := jwtProvider.Validate(ctx, tokenString)
if err != nil {
    return fmt.Errorf("invalid token: %w", err)
}
fmt.Println("Subject:", claims.Subject())
```

11.2 digital.vasic/database

PostgreSQL connection pooling, migration execution, and query helpers.

```
import "digital.vasic/database"

// Initialization (in main.go)
dbPool, err := database.Connect(ctx, database.Config{
    Host:      cfg.Database.Host,
    Port:      cfg.Database.Port,
    User:      cfg.Database.User,
    Password:  cfg.Database.Password,
    Database:  cfg.Database.Name,
    MaxConns:  cfg.Database.MaxConnections,
    MinConns:  cfg.Database.MinConnections,
})
if err != nil {
    logger.Error("failed to connect to database", "error", err)
    os.Exit(1)
}
defer dbPool.Close()

// Migration
migrator := database.NewMigrator(dbPool, "migrations")
if err := migrator.Up(ctx); err != nil {
    logger.Error("failed to run migrations", "error", err)
    os.Exit(1)
}

// Usage: Query with scan
var device model.Device
err = database.QueryRow(ctx, dbPool,
    "SELECT id, name, status, hardware_rev FROM devices WHERE
    id = $1",
    deviceID,
).Scan(&device.ID, &device.Name, &device.Status,
    &device.HardwareRevision)
```

11.3 digital.vasic/cache

Redis client wrapper with typed operations and distributed locking.

```
import "digital.vasic/cache"

// Initialization (in main.go)
```



```

redisClient, err := cache.Connect(ctx, cache.Config{
    Addr:      fmt.Sprintf("%s:%d", cfg.Redis.Host,
        cfg.Redis.Port),
    Password:  cfg.Redis.Password,
    DB:        cfg.Redis.DB,
})
if err != nil {
    logger.Error("failed to connect to Redis", "error", err)
    os.Exit(1)
}
defer redisClient.Close()

// Usage: Set with TTL
err = redisClient.Set(ctx, "heartbeat:device-123",
    time.Now().Unix(), 0)

// Usage: Get with type safety
lastSeen, err := redisClient.GetInt64(ctx, "heartbeat:device-123")

// Usage: Atomic SETNX for distributed locking
acquired, err := redisClient.SetNX(ctx,
    "device_assignment:device-123", "rollout-456",
    24*time.Hour)

// Usage: Scan keys by pattern
keys, err := redisClient.ScanKeys(ctx, "heartbeat:*")

```

11.4 digital.vasic/observability

Structured logging, metrics, and tracing.

```

import "digital.vasic/observability"

// Initialization (in main.go)
logger := observability.NewLogger(observability.Config{
    Level:  cfg.LogLevel,
    Format: cfg.LogFormat, // "json" or "text"
})

// Usage: Structured logging
logger.Info("device registered",
    "device_id", device.ID,
    "hardware_rev", device.HardwareRevision,
    "group", device.Group,
)

logger.Warn("rollout failure rate elevated",
    "rollout_id", rollout.ID,
    "failure_rate", fmt.Sprintf("%.2f%%", failureRate*100),
)

logger.Error("database connection lost",
    "error", err,

```

```

    "host", cfg.Database.Host,
)

// Usage: Counters and histograms
counter := observability.NewCounter("events_ingested_total")
counter.Inc()

histogram :=
    observability.NewHistogram("request_duration_seconds")
histogram.Observe(0.42)

```

11.5 digital.vasic/security

Cryptography utilities: certificate management, key rotation, and secure random generation.

```

import "digital.vasic/security"

// Usage: Generate secure random device token
token, err := security.GenerateRandomString(32)
if err != nil {
    return fmt.Errorf("generate token: %w", err)
}

// Usage: Hash password with bcrypt
hashedPassword, err := security.HashPassword("user-password")
if err != nil {
    return fmt.Errorf("hash password: %w", err)
}

// Usage: Verify password
match, err := security.VerifyPassword(hashedPassword, "user-
password")

// Usage: Parse and validate X.509 certificate
cert, err := security.ParseCertificatePEM(certPEM)
if err != nil {
    return fmt.Errorf("parse certificate: %w", err)
}

// Usage: Verify certificate chain
err = security.VerifyCertificateChain(cert, caCertPool)

```

11.6 digital.vasic/middleware

HTTP middleware components: CORS, compression, and request tracing.

```

import (
    "digital.vasic/middleware/cors"
    "digital.vasic/middleware/compress"
)

```

```

// Usage: CORS middleware
corsHandler := cors.Handler(cors.Config{
    AllowedOrigins:  []string{"https://admin.helix-ota.io"},
    AllowedMethods:  []string{"GET", "POST", "PUT", "DELETE",
        "OPTIONS"},
    AllowedHeaders:  []string{"Authorization", "Content-Type",
        "X-Request-ID"},
    AllowCredentials: true,
    MaxAge:          3600,
})

// Usage: Response compression
compressHandler := compress.Handler(compress.Config{
    Level: compress.DefaultCompression,
    Types: []string{"application/json", "text/html"},
})

// Apply to router
router.Use(corsHandler)
router.Use(compressHandler)

```

11.7 digital.vasic/config

Multi-source configuration loading: environment variables, YAML files, and defaults.

```

import "digital.vasic/config"

// Initialization (in main.go)
loader := config.NewLoader(config.ConfigOptions{
    EnvPrefix: "HELIX_OTA",    // Maps HELIX_OTA_SERVER_PORT to
        Server.Port
    Files:      []string{"config.yaml", "/etc/helix-ota/
        config.yaml"},
    Required:   []string{"database.host", "redis.host"},
})

var cfg ServerConfig
if err := loader.Load(&cfg); err != nil {
    return nil, fmt.Errorf("load configuration: %w", err)
}

// Configuration precedence (highest to lowest):
// 1. Environment variables (HELIX_OTA_*)
// 2. config.yaml file
// 3. Default values applied in code

```

11.8 digital.vasic/eventbus

In-process event bus with topic-based pub/sub, fan-out, and ordered delivery.

```

import "digital.vasic/eventbus"

// Initialization (in main.go)
bus := eventbus.New(eventbus.Config{
    BufferSize: 1000, // Per-subscriber buffer
    Workers:   4,    // Concurrent event dispatchers
})
defer bus.Close()

// Usage: Publish an event
bus.Publish(ctx, eventbus.Event{
    Topic:   "telemetry.event",
    Key:     deviceID, // Key for partitioning
    Payload: telemetryEvent,
})

// Usage: Subscribe to a topic
bus.Subscribe("telemetry.event", func(ctx context.Context, event
    interface{}) {
    te := event.(*model.TelemetryEvent)
    // Process event
    logger.Info("received telemetry event",
        "device_id", te.DeviceID,
        "event_type", te.EventType,
    )
})

// Usage: Subscribe with filter
bus.SubscribeFiltered("rollout.halted", eventbus.Filter{
    Key: "rollout_id",
    Match: "rollout-123",
}, handler)

```

11.9 digital.vasic/storage

Object storage abstraction supporting S3, GCS, and local filesystem.

```

import "digital.vasic/storage"

// Initialization (in main.go)
storageClient, err := storage.Connect(ctx, storage.Config{
    Provider: cfg.Storage.Provider, // "s3", "gcs", or "local"
    Bucket:   cfg.Storage.Bucket,
    Region:   cfg.Storage.Region,
    Endpoint: cfg.Storage.Endpoint, // For MinIO/LocalStack
})
if err != nil {
    logger.Error("failed to connect to storage", "error", err)
    os.Exit(1)
}

// Usage: Upload artifact

```

```

err = storageClient.Upload(ctx, "artifacts/firmware-v2.1.0.bin",
    artifactReader,
    storage.UploadOptions{
        ContentType: "application/octet-stream",
        Metadata: map[string]string{
            "version":      "2.1.0",
            "hardware":     "rev-c",
            "uploaded_by": "release-pipeline",
        },
    },
)

// Usage: Generate signed download URL (time-limited)
downloadURL, err := storageClient.SignedURL(ctx, "artifacts/
    firmware-v2.1.0.bin",
    storage.SignedURLOptions{
        Expiry: 1 * time.Hour,
        Method: "GET",
    },
)

// Usage: Delete artifact
err = storageClient.Delete(ctx, "artifacts/firmware-v2.1.0.bin")

// Usage: Check existence
exists, err := storageClient.Exists(ctx, "artifacts/firmware-
    v2.1.0.bin")

```

11.10 digital.vasic/ratelimiter

Distributed rate limiting backed by Redis with sliding window algorithm.

```

import "digital.vasic/ratelimiter"

// Initialization (in middleware chain)
store := ratelimiter.NewRedisStore(redisClient)

limiter := ratelimiter.Handler(ratelimiter.Config{
    Rate:      100,    // 100 requests per second
    Burst:     200,    // Allow bursts up to 200
    TTL:       1 * time.Minute,
    KeyFunc:   ratelimiter.IPKeyFunc, // Rate limit per IP
    Store:     store,
    OnLimited: func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Retry-After", "60")
        http.Error(w,
            `{"error":"rate_limit_exceeded","retry_after":60}`,
            http.StatusTooManyRequests)
    },
})

// Apply to router
router.Use(limiter)

```

```
// Usage: Custom key function for per-device rate limiting
deviceKeyFunc := func(r *http.Request) string {
    if deviceID := middleware.GetDeviceID(r.Context()); deviceID != "" {
        return "device:" + deviceID
    }
    return "ip:" + r.RemoteAddr
}
```

11.11 digital.vasic/concurrency

Goroutine pool, errgroup with limits, and synchronization primitives.

```
import "digital.vasic/concurrency"

// Usage: Bounded goroutine pool for batch validation
g, ctx := concurrency.NewGroup(ctx,
    concurrency.WithMaxGoroutines(8))

for _, artifact := range artifacts {
    artifact := artifact
    g.Go(func() error {
        return pipeline.Validate(ctx, artifact, reader)
    })
}

if err := g.Wait(); err != nil {
    logger.Error("batch validation failed", "error", err)
}

// Usage: Semaphore for limiting concurrent artifact downloads
downloadLimiter := concurrency.NewSemaphore(10) // Max 10
           concurrent downloads

err := downloadLimiter.Acquire(ctx)
if err != nil {
    return err
}
defer downloadLimiter.Release()

// Perform download...

// Usage: Singleflight for deduplicating concurrent requests for
           the same artifact
sf := concurrency.NewSingleflight()
result, err, shared := sf.Do(ctx, artifactID, func()
    (interface{}, error) {
        return fetchArtifactMetadata(ctx, artifactID)
    })
}
```

11.12 digital.vasic/recovery

Panic recovery for HTTP handlers and background jobs.

```
import "digital.vasic/recovery"

// Usage: HTTP middleware (applied in middleware chain)
recoveryMiddleware := recovery.HTTPHandler(recovery.Config{
    LogStack:    true,
    Logger:      logger,
    DefaultError: `{"error":"internal_server_error"}`,
    OnPanic: func(r *http.Request, recovered interface{}) {
        // Increment panic counter metric
        panicCounter.Inc()
        // Send alert to on-call
        alertManager.Send("server panic detected", recovered)
    },
})

router.Use(recoveryMiddleware)

// Usage: Wrap background job handler
safeHandler := recovery.WrapJob(jobHandler, recovery.Config{
    LogStack: true,
    Logger:   logger,
})
```

11.13 digital.vasic/backgroundtasks

Scheduled task execution with retry, monitoring, and graceful shutdown.

```
import "digital.vasic/backgroundtasks"

// Initialization (in main.go)
scheduler :=
    backgroundtasks.NewScheduler(backgroundtasks.SchedulerConfig{
        Logger: logger,
    })

// Register tasks
err := scheduler.Register(backgroundtasks.TaskConfig{
    Name:        "rollout_evaluation",
    Description: "Evaluate active rollouts and advance/halt as needed",
    Interval:    1 * time.Minute,
    Timeout:     5 * time.Minute,
    MaxRetries:  3,
    RetryDelay:  30 * time.Second,
    Handler: func(ctx context.Context) error {
        return rolloutEngine.EvaluateAll(ctx)
    },
})
```

```
// Start the scheduler
scheduler.Start()

// Graceful shutdown (blocks until all in-progress tasks complete
// or timeout)
scheduler.Stop()

// Usage: Get task status
status := scheduler.Status("rollout_evaluation")
fmt.Printf("Last run: %s, Next run: %s, Success: %v\n",
    status.LastRun, status.NextRun, status.LastSuccess)

// Usage: Run a one-off task
err := scheduler.RunOnce(ctx, backgroundtasks.TaskConfig{
    Name:      "emergency_rollout_halt",
    Timeout:   30 * time.Second,
    Handler:   func(ctx context.Context) error {
        return rolloutEngine.HaltAll(ctx, "emergency maintenance")
    },
})
```

Appendix A: SQL Migrations

001_create_devices.up.sql

```
CREATE TABLE devices (
    id          VARCHAR(64) PRIMARY KEY,
    name        VARCHAR(255) NOT NULL,
    hardware_rev VARCHAR(64) NOT NULL,
    current_version VARCHAR(32) NOT NULL DEFAULT '',
    device_group VARCHAR(128) NOT NULL DEFAULT 'default',
    status      VARCHAR(32) NOT NULL DEFAULT 'ACTIVE',
    fingerprint JSONB      NOT NULL,
    certificate_sn VARCHAR(128),
    last_seen_at TIMESTAMPTZ,
    tags        JSONB      NOT NULL DEFAULT '{}',
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_devices_status ON devices(status);
CREATE INDEX idx_devices_group ON devices(device_group);
CREATE INDEX idx_devices_hw_rev ON devices(hardware_rev);
CREATE INDEX idx_devices_last_seen ON devices(last_seen_at);
```

004_create_rollouts.up.sql

```
CREATE TABLE rollouts (
    id          VARCHAR(64) PRIMARY KEY,
```



```

update_id          VARCHAR(64) NOT NULL REFERENCES
    updates(id),
artifact_id        VARCHAR(64) NOT NULL REFERENCES
    artifacts(id),
name               VARCHAR(255) NOT NULL,
description         TEXT,
status             VARCHAR(32) NOT NULL DEFAULT 'DRAFT',
target_group       VARCHAR(128) NOT NULL,
hardware_rev       VARCHAR(64) NOT NULL,
stages             JSONB NOT NULL,
current_stage_index INTEGER NOT NULL DEFAULT 0,
failure_threshold  DOUBLE PRECISION NOT NULL DEFAULT 0.1,
min_dwelling_duration INTERVAL NOT NULL DEFAULT '30
    minutes',
stage_entered_at   TIMESTAMPTZ,
halted_reason      TEXT,
created_by         VARCHAR(64) NOT NULL,
created_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
updated_at         TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_rollouts_status ON rollouts(status);
CREATE INDEX idx_rollouts_group ON rollouts(target_group,
    hardware_rev);

```

Appendix B: Error Handling Strategy

All errors in the Helix OTA server follow a consistent pattern:

1. **Repository layer** returns raw errors wrapped with `fmt.Errorf("operation: %w", err)`.
2. **Service layer** translates repository errors into domain errors from `internal/model/errors.go`.
3. **Handler layer** maps domain errors to HTTP status codes using a centralized mapper.

```

// internal/model/errors.go
package model

import "errors"

var (
    ErrDeviceNotFound          = errors.New("device not found")
    ErrDeviceAlreadyExists    = errors.New("device already exists")
    ErrUpdateNotFound          = errors.New("update not found")
    ErrRolloutNotFound         = errors.New("rollout not found")
    ErrRolloutNotMutable       = errors.New("rollout is not in a
        mutable state")
    ErrArtifactNotFound        = errors.New("artifact not found")
    ErrArtifactInvalid         = errors.New("artifact validation
        failed")
    ErrUnauthorized            = errors.New("unauthorized")

```

```

ErrForbidden          = errors.New("insufficient
    permissions")
ErrConflict            = errors.New("resource conflict")
)

// internal/handler/error_mapper.go
package handler

import (
    "net/http"

    "github.com/vasic-digital/helix-ota-server/internal/model"
    "github.com/vasic-digital/helix-ota-server/pkg/api"
)

func mapError(err error) (int, *api.APIError) {
    switch {
    case errors.Is(err, model.ErrDeviceNotFound),
        errors.Is(err, model.ErrUpdateNotFound),
        errors.Is(err, model.ErrRolloutNotFound),
        errors.Is(err, model.ErrArtifactNotFound):
        return http.StatusNotFound, &api.APIError{
            Code:    "NOT_FOUND",
            Message: err.Error(),
        }
    case errors.Is(err, model.ErrDeviceAlreadyExists),
        errors.Is(err, model.ErrConflict):
        return http.StatusConflict, &api.APIError{
            Code:    "CONFLICT",
            Message: err.Error(),
        }
    case errors.Is(err, model.ErrRolloutNotMutable):
        return http.StatusConflict, &api.APIError{
            Code:    "INVALID_STATE",
            Message: err.Error(),
        }
    case errors.Is(err, model.ErrArtifactInvalid):
        return http.StatusBadRequest, &api.APIError{
            Code:    "VALIDATION_FAILED",
            Message: err.Error(),
        }
    case errors.Is(err, model.ErrUnauthorized):
        return http.StatusUnauthorized, &api.APIError{
            Code:    "UNAUTHORIZED",
            Message: err.Error(),
        }
    case errors.Is(err, model.ErrForbidden):
        return http.StatusForbidden, &api.APIError{
            Code:    "FORBIDDEN",
            Message: err.Error(),
        }
    default:
        return http.StatusInternalServerError, &api.APIError{

```

```
        Code:      "INTERNAL_ERROR",
        Message:    "An unexpected error occurred",
    }
}
```

Appendix C: Configuration Reference

All configuration is loaded via `digital.vasic/config` with the environment variable prefix `HELIX_OTA_`. Nested fields use double underscores (e.g., `HELIX_OTA_DATABASE__HOST`).

Environment Variable	YAML Path	Type	Default
<code>HELIX_OTA_SERVER__PORT</code>	<code>server.port</code>	int	8080
<code>HELIX_OTA_SERVER__READ_TIMEOUT</code>	<code>server.read_timeout</code>	duration	15s
<code>HELIX_OTA_SERVER__WRITE_TIMEOUT</code>	<code>server.write_timeout</code>	duration	60s
<code>HELIX_OTA_DATABASE__HOST</code>	<code>database.host</code>	string	localhost
<code>HELIX_OTA_DATABASE__PORT</code>	<code>database.port</code>	int	5432
<code>HELIX_OTA_DATABASE__NAME</code>	<code>database.name</code>	string	helix_ota
<code>HELIX_OTA_DATABASE__MAX_CONNECTIONS</code>	<code>database.max_connections</code>	int	25
<code>HELIX_OTA_REDIS__HOST</code>	<code>redis.host</code>	string	localhost
<code>HELIX_OTA_REDIS__PORT</code>	<code>redis.port</code>	int	6379
<code>HELIX_OTA_AUTH__ACCESS_TOKEN_TTL</code>	<code>auth.access_token_ttl</code>	duration	15m
<code>HELIX_OTA_AUTH__REFRESH_TOKEN_TTL</code>	<code>auth.refresh_token_ttl</code>	duration	168h
<code>HELIX_OTA_RATE_LIMIT__RPS</code>	<code>rate_limit.rps</code>	float	100
<code>HELIX_OTA_RATE_LIMIT__BURST</code>	<code>rate_limit.burst</code>	int	200
<code>HELIX_OTA_STORAGE__PROVIDER</code>	<code>storage.provider</code>	string	s3
<code>HELIX_OTA_STORAGE__BUCKET</code>	<code>storage.bucket</code>	string	helix-ota-artifacts
<code>HELIX_OTA_LOG_LEVEL</code>	<code>log_level</code>	string	info
<code>HELIX_OTA_RUN_MIGRATIONS</code>	<code>run_migrations</code>	bool	false

Environment Variable	YAML Path	Type	Default

This document specifies the complete Go server implementation design for the Helix OTA system. All code examples are production-ready patterns that follow Go best practices for error handling, concurrency, and dependency injection. Implementation should proceed in the order: models → repositories → services → handlers → main, with tests written alongside each layer.